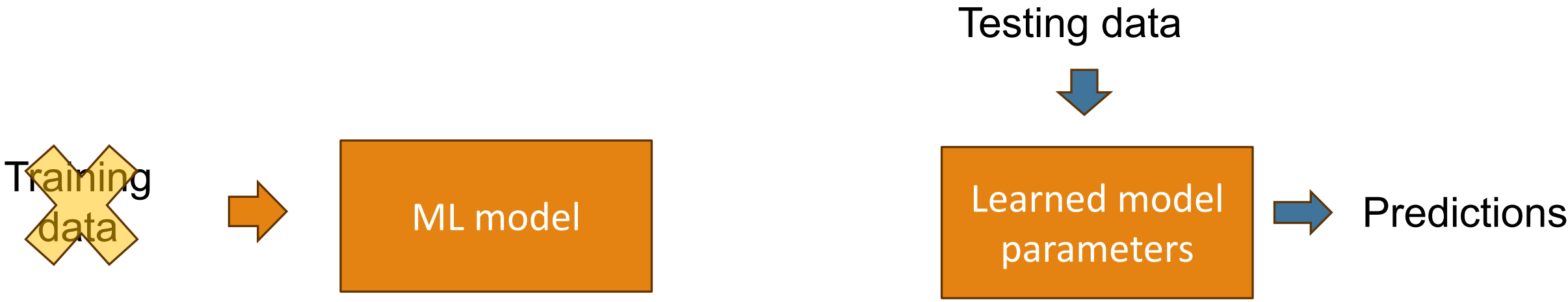
EN3150 Pattern Recognition
Kernel Methods

Most of the contents are based on

1. C. M. Bishop, Pattern Recognition and Machine Learning, Springer NY, 2006.
2. Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.
3. Shawe-Taylor, John, and Nello Cristianini. Kernel methods for pattern analysis. Cambridge university press, 2004.
4. CS229 Lecture notes Dr. Andrew Ng
5. Density Estimation — Applied ML (kuleshov-group.github.io)

Presented by Sampath K. Perera

Recap



- Training data is used to learn the parameters of the ML model.
- Once the parameter estimation is complete, the training data is no longer needed.
- Predictions for new inputs rely solely on the learned parameter values and in the testing phase training data is not used*.

*e.g., linear regression, neural networks

C. M. Bishop, Pattern Recognition and Machine Learning, Springer NY, 2006

Memory-based methods

- In some pattern recognition techniques, training data points, or a subset, are retained and utilized during predictions (testing phase) as well.
- Memory-based methods, E.g., Locally weighted regression, k-Nearest Neighbors (k-NN)
- Typically, a similarity metric is needed to measure the likeness between any two vectors in the input space → eg: Euclidean distance, cosine similarity
- Training is usually fast
- Prediction for test data points tends to be slower.

training fast, prediction slow

The Representer Theorem

data loss

regularization loss

$$J(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} L(\mathbf{y}_i, \mathbf{x}_i, \mathbf{w}) + r(\mathbf{w})$$

$$\text{e.g., } J(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} (y_i - \mathbf{w}^T\mathbf{x}_i)^2 + \lambda\|\mathbf{w}\|_2^2$$

$$\mathbf{w}* = \frac{1}{N}\sum_{i=1}^{N} \alpha_i \mathbf{x}_i$$

empirical loss = average loss over training set

- In certain cases, the solution to an empirical risk minimization problem (e.g., training a model to fit data) can be expressed as a linear combination of the training data points.
- When dealing with high-dimensional feature spaces, optimization can be computationally intensive. The theorem reduces this complexity, where inner product can be calculated efficiently.

Representer theorem - Wikipedia

representer-function(stanford.edu)

Dual representation

linear classifiers like linear regression, logistic regression

- Many linear parametric models can be re-cast into an equivalent "dual representation" in which the predictions are also based on linear combinations of a kernel function evaluated at the training data points.

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot \sum_{i=1}^{N} \left(y_i - \mathbf{w}^T \mathbf{x}_i^{train}\right) \mathbf{x}_i^{train}$$

$$y_j = \mathbf{w}^T \mathbf{x}_j^{test}$$

Learned $\mathbf{w}$ are linear combinations of training data points

predictions are also based on linear combinations of training data points

kernel trick: $K(\mathbf{x}, \acute{\mathbf{x}}) = \phi(\mathbf{x})^T \phi(\acute{\mathbf{x}})$

Ridge Regression: Recap

Ridge Regression

$$J(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \mathbf{w}^T\mathbf{x}_i)^2 + \lambda\|\mathbf{w}\|_2^2$$

MSE loss

L_2 regularization

$$\hat{\mathbf{w}}_{\text{ridge}} = (\lambda\mathbf{I}_D + \mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

closed form solution

Polynomial Ridge Regression

$$J(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \mathbf{w}^T\phi(\mathbf{x}_i))^2 + \lambda\|\mathbf{w}\|_2^2$$

For single variable $\phi(\mathbf{x}) = [1, x, x^2, \ldots, x^d]$ $\quad \hat{y}(\mathbf{x}) = w_0 + w_1 x + w_2 x^2 \ldots, + w_D x^d$

For two features $d = 2$, $\phi(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_1x_2, {x_2}^2]$ $\quad \hat{y}(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_1x_2 + w_5 x_2^2$

Kernelizing linear regression

$y = w^T x$ (linear)

$y = w^T \phi(x)$ (polynomial kernel)

$w = (\phi^T \phi)^{-1} \phi^T y$ (w/o reg)

$w = (\phi^T \phi + \lambda I)^{-1} \phi^T y$ (w/ reg)

} without kernalization

ridge regression

covariance matrix

Kernalizing...

$$J(w) = \sum_{i=1}^{n} (y_n - w^T \phi(x_n))^2 + \frac{\lambda}{2} \sum_{i=1}^{n} \|w\|^2$$

$$\frac{\partial J}{\partial w} = 0 \Rightarrow w^* = \frac{1}{\lambda} \sum_{i=1}^{n} \underbrace{(y_n - w^T \phi(x_n))}_{\alpha_n} \phi(x)$$

new w ↗

$$w^* = \sum_{i=1}^{n} \alpha_n \phi(x_n)$$ ← dot product of ϕ and α

$$\Rightarrow \quad w^* = \phi^T \alpha$$

$$J(w) = \sum_{i=1}^{n} (y - w^T \phi(x_n))^2 + \frac{\lambda}{2} \sum_{i=1}^{n} \|w\|^2 = (y - \phi w)^T (y - \phi w) + \frac{\lambda}{2} w^T w$$

$$= y^T y - y^T \phi w - w^T \phi^T y + w^T \phi^T \phi w + \frac{\lambda}{2} w^T w$$

$$= y^T y - y^T \phi(\phi^T \alpha) - (\phi^T \alpha)^T \phi^T y + (\phi^T \alpha)^T \phi^T \phi(\phi^T \alpha) + \frac{\lambda}{2} (\phi^T \alpha)^T (\phi^T \alpha)$$

$$= y^T y - y^T (\phi \phi^T) \alpha - \alpha^T (\phi \phi^T) y + \alpha^T (\phi \phi^T)(\phi \phi^T) \alpha + \frac{\lambda}{2} \alpha^T (\phi \phi^T) \alpha$$

Gram / kernel matrix ↑

$$\boldsymbol{\phi}\boldsymbol{\phi}^T = K = \begin{bmatrix} \phi(x_1)^T\phi(x_1) & \phi(x_1)^T\phi(x_2) & . & \phi(x_1)^T\phi(x_n) \\ \phi(x_2)^T\phi(x_1) & \phi(x_2)^T\phi(x_2) & . & . \\ . & . & . & . \\ \phi(x_m)^T\phi(x_1) & \phi(x_m)^T\phi(x_2) & . & \phi(x_m)^T\phi(x_n) \end{bmatrix}$$

Kernel matrix

* symmetric ($K = K^T$)
* positive semi-definite
  $\alpha^T K \alpha \geq 0$

using these properties ↓

$$J(\alpha) = y^T y - y^T K\alpha - y^T K\alpha + \alpha^T K^2 \alpha + \frac{\lambda}{2}\alpha^T K \alpha = y^T y - 2y^T K\alpha + \alpha^T K^2 \alpha + \frac{\lambda}{2}\alpha^T K\alpha$$

$$\frac{\partial J(\alpha)}{\partial \alpha} = 0 - 2y^T K + 2\alpha^T K^2 + \lambda \alpha^T K = 0 \Rightarrow \alpha = \left(K + \frac{\lambda}{2} I\right)^{-1} y$$

$$w = \phi^T \alpha = \phi^T (K + \lambda I)^{-1} y = \phi^T (\phi\phi^T + \lambda I)^{-1} y \quad \text{(kernalized)}$$

kernel matrix

still need ϕ?

↓ no need (Mercer's theorem)

psd matrix

$$w = (\phi^T \phi + \lambda I)\phi^T y \quad \text{(not kernalized)}$$

covariance matrix

$$\boldsymbol{\phi}\boldsymbol{\phi}^T = K = \begin{bmatrix} \phi(x_1)^T\phi(x_1) & \phi(x_1)^T\phi(x_2) & . & \phi(x_1)^T\phi(x_n) \\ \phi(x_2)^T\phi(x_1) & \phi(x_2)^T\phi(x_2) & . & . \\ . & . & . & . \\ \phi(x_m)^T\phi(x_1) & \phi(x_m)^T\phi(x_2) & . & \phi(x_m)^T\phi(x_n) \end{bmatrix} = \begin{bmatrix} k(x_1,x_1) & k(x_1,x_2) & . & k(x_1,x_n) \\ k(x_2,x_1) & k(x_2,x_2) & . & . \\ . & . & . & . \\ k(x_m,x_1) & k(x_m,x_2) & . & k(x_m,x_n) \end{bmatrix}$$

represent inner products as a function of input variables

Dual representation

- $J(\mathbf{w}) = \frac{1}{2}\sum_{i=1}^{N}(\mathbf{w}^T\phi(\mathbf{x}_i) - y_i)^2 + \frac{1}{2}\lambda\|\mathbf{w}\|_2^2$ = $\frac{1}{2}\sum_{i=1}^{N}(\mathbf{w}^T\phi(\mathbf{x}_i) - y_i)^2 + \frac{1}{2}\lambda\ \mathbf{w}^T\mathbf{w}$, and $\lambda \geq 0$
- $\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = 0$, ➔ $\mathbf{w} = \sum_{i=1}^{N}\frac{-1}{\lambda}\ (\mathbf{w}^T\phi(\mathbf{x}_i) - y_i)\ \phi(\mathbf{x}_i)$.
- $\mathbf{w} = \sum_{i=1}^{N} a_i\,\phi(\mathbf{x}_i)$ with $a_i = \frac{-1}{\lambda}\ \{\mathbf{w}^T\phi(\mathbf{x}_i) - y_i\}$.
- $\mathbf{w} = \boldsymbol{\phi}^T\mathbf{a}$ and i-th row of the $\boldsymbol{\phi}$ is $\phi(\mathbf{x}_i)^T$ and $\mathbf{a} = (a_1, \ldots, a_N)^T$
- Substitute $\mathbf{w} = \boldsymbol{\phi}^T\mathbf{a}$ to $J(\mathbf{w}) = \frac{1}{2}\sum_{i=1}^{N}(\mathbf{w}^T\phi(\mathbf{x}_i) - y_i)^2 + \frac{1}{2}\lambda\ \mathbf{w}^T\mathbf{w}$
- $J(\mathbf{a}) = \frac{1}{2}\mathbf{a}^T\boldsymbol{\phi}\,\boldsymbol{\phi}^T\,\boldsymbol{\phi}\,\boldsymbol{\phi}^T\mathbf{a} - \mathbf{a}^T\boldsymbol{\phi}\,\boldsymbol{\phi}^T\mathbf{y} + \frac{1}{2}\mathbf{y}^T\mathbf{y} + \frac{\lambda}{2}\mathbf{a}^T\boldsymbol{\phi}\,\boldsymbol{\phi}^T\mathbf{a}$. Here, $\mathbf{y} = (y_1, \ldots, y_N)^T$.
- Define Gram matrix $\boldsymbol{K} = \boldsymbol{\phi}\,\boldsymbol{\phi}^T$ ➔ $J(\mathbf{a}) = \frac{1}{2}\mathbf{a}^T\boldsymbol{K}\boldsymbol{K}\mathbf{a} - \mathbf{a}^T\boldsymbol{K}\mathbf{y} + \frac{1}{2}\mathbf{y}^T\mathbf{y} + \frac{\lambda}{2}\mathbf{a}^T\boldsymbol{K}\mathbf{a}$
- Setting the gradient of $J(\mathbf{a})$ with respect to $\mathbf{a}$ to zero ➔ $\mathbf{a} = (\lambda\mathbf{I}_N + \mathbf{K})^{-1}\mathbf{y}$
- Prediction for a new input $\acute{\mathbf{x}}$ is given by $\acute{y} = \mathbf{w}^T\phi(\acute{\mathbf{x}})$
- $\mathbf{w} = \boldsymbol{\phi}^T\mathbf{a}$ (from training data), we get $\acute{y} = (\boldsymbol{\phi}^T\mathbf{a})^T\phi(\acute{\mathbf{x}}) = \boldsymbol{a}^T\ \boldsymbol{\phi}\ \phi(\acute{\mathbf{x}})$
- $\mathbf{a} = (\lambda\mathbf{I}_N + \mathbf{K})^{-1}\mathbf{y}$, we get $\acute{y} = \boldsymbol{K}(\acute{\mathbf{x}})^T(\lambda\mathbf{I}_N + \mathbf{K})^{-1}\mathbf{y}$
- $i-$ th element of $\boldsymbol{K}(\acute{\mathbf{x}})$ is given by $\boldsymbol{K}_i(\acute{\mathbf{x}}) = \boldsymbol{K}(\mathbf{x}_i, \acute{\mathbf{x}})$

Gram matrix
$\boldsymbol{K}_{ij} = \phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j) = k(\mathbf{x}_i, \mathbf{x}_j)$
kernel function $k(\mathbf{x}_i, \mathbf{x}_j)$

kernel trick

function of base features

inner product

- **kernel trick:** $K(\mathbf{x}, \acute{\mathbf{x}}) = \phi(\mathbf{x})^T \phi(\acute{\mathbf{x}})$
- The kernel concept, initially introduced by Aizerman et al. (1964) in the method of potential functions, was largely overlooked for years.
- It was introduced to machine learning when Boser et al. (1992) reintroduced it in the development of large-margin classifiers, leading to support vector machines.

$$\sum_i \alpha_i y_i \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}) \; + b = \sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) \; + b$$

Kernel function that returns the value of the dot product between $\mathbf{x}_i$ and $\mathbf{x}$

kernel trick

- Kernel Concept: Utilizes inner products in feature space.
- Kernel Trick: Enhances existing algorithms by replacing scalar products with alternative kernels.
- $\mathbf{x}^T\acute{\mathbf{x}} \Rightarrow K(\mathbf{x}, \acute{\mathbf{x}}) = \phi(\mathbf{x})^T\phi(\acute{\mathbf{x}})$
- Applicable to algorithms where input vectors are used as scalar products.
  - E.g., Nonlinear PCA, nearest-neighbour classifiers and the kernel Fisher discriminant

example

$$\phi: x \to \phi(x) = \begin{bmatrix} x_1^2 \\ \sqrt{2} x_1 x_2 \\ x_2^2 \end{bmatrix}$$

$$\phi(x_m)^T \phi(x_n) = \begin{bmatrix} x_{m1}^2 \\ \sqrt{2} x_{m1} x_{m2} \\ x_{m2}^2 \end{bmatrix}^T \begin{bmatrix} x_{n1}^2 \\ \sqrt{2} x_{n1} x_{n2} \\ x_{n2}^2 \end{bmatrix} = x_{m1}^2 x_{n1}^2 + 2 x_{m1} x_{m2} x_{n1} x_{n2} + x_{n2}^2 x_{m2}^2$$

$$= (x_{m1} x_{n1} + x_{m2} x_{n2})^2$$

$$= (x_m^T x_n)^2$$

$$= K(x_m, x_n) \sim (x_m^T x_n + r)^d$$

polynomial kernel

Make predictions?

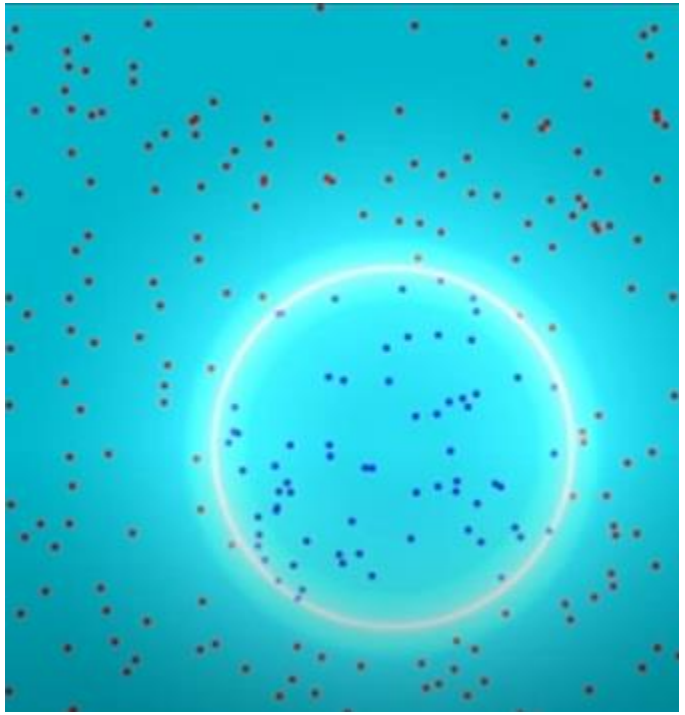
$$y = w^T \phi(x) = y(\phi \phi^T + \lambda I)^{-1} \underbrace{\phi^T \phi(x)}_{k_x} = y(K + \lambda I) k_x$$

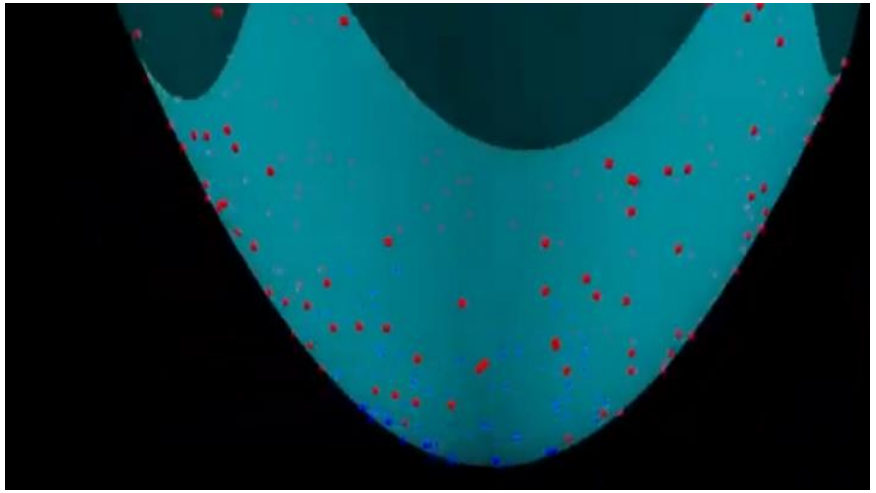
independent of ϕ

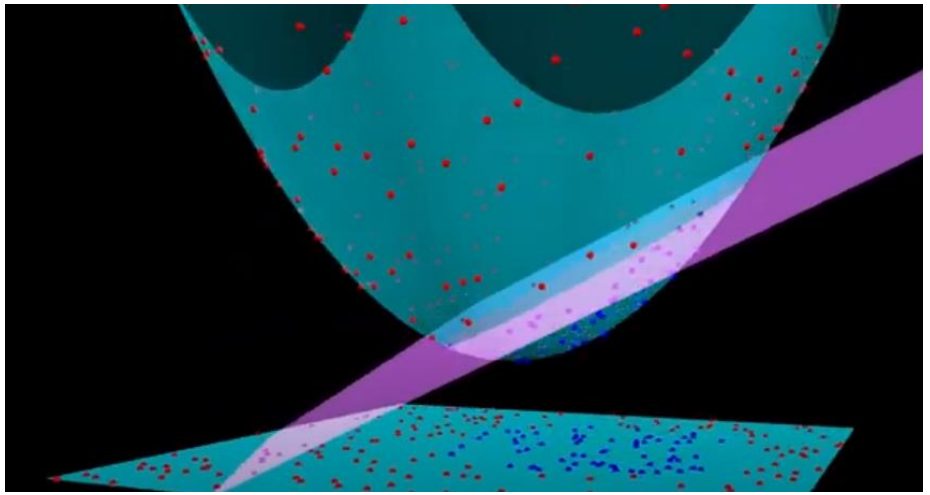
Kernel trick

SVM with polynomial kernel visualization – YouTube

$$k(x,y) = [x, y, (x^2+y^2)]$$

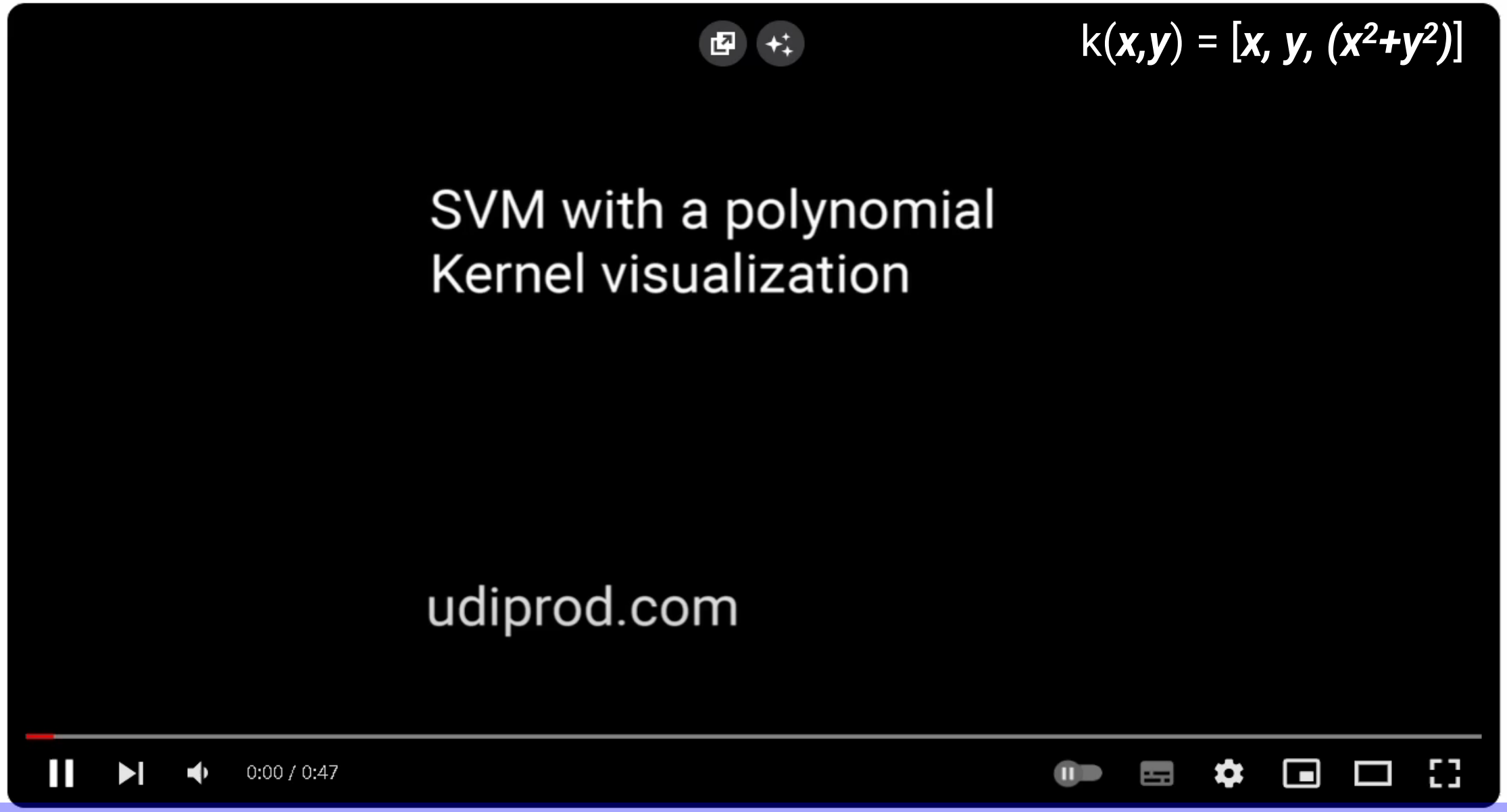






visualization from www.udiprod.com

Kernel trick



visualization from www.udiprod.com

Input space to feature space

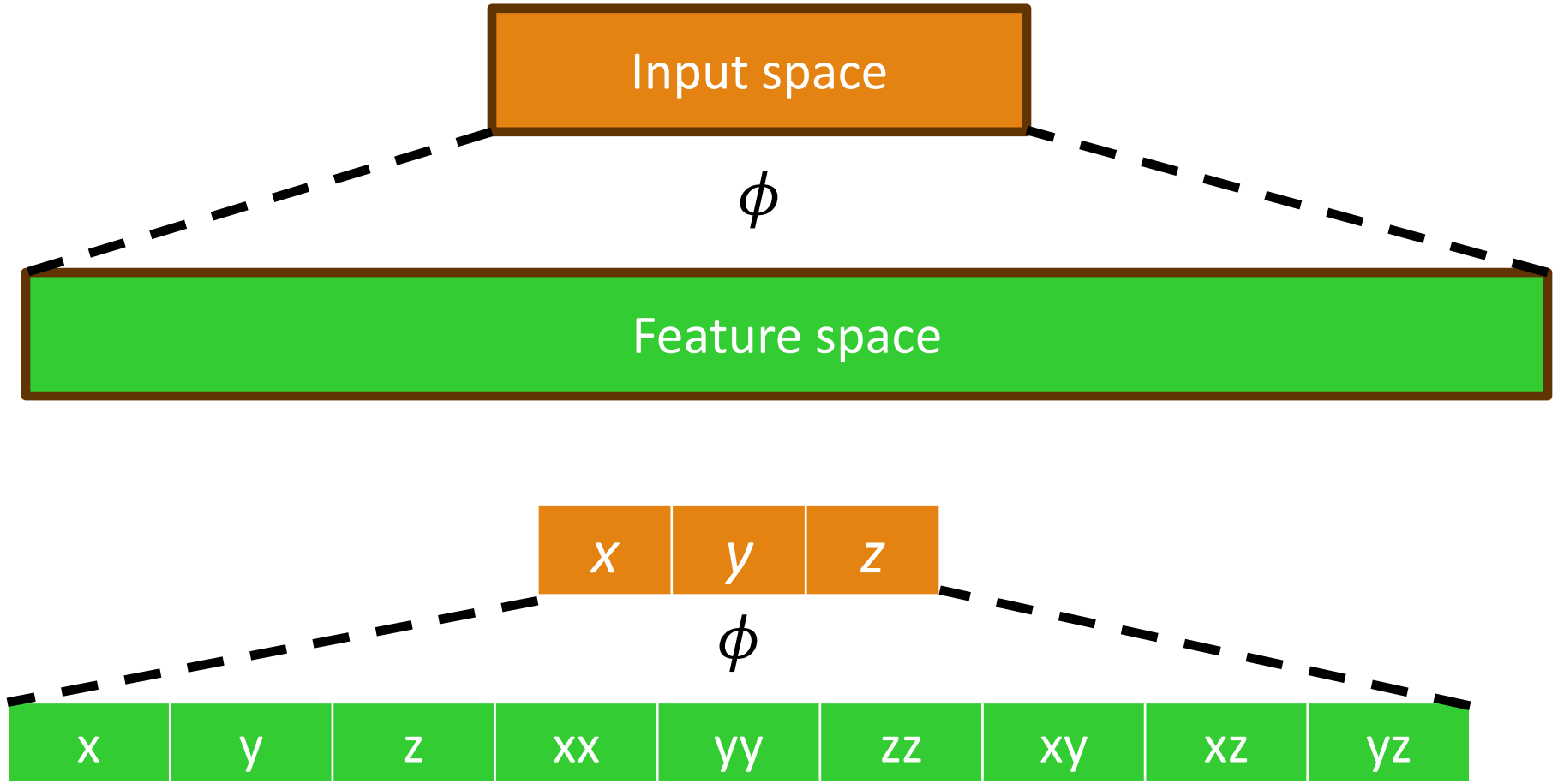


Image source Support Vector Machines

Input space to feature space

- Two − dimensional input space $\boldsymbol{x} \subseteq \mathbb{R}^2$ with feature map

$$\phi: \boldsymbol{x} = (x_1, x_2) \rightarrow \phi(\boldsymbol{x}) = (x_1^2, x_2^2, \sqrt{2}x_1x_2) \in F = \mathbb{R}^3.$$

- The hypothesis space of linear functions in feature space (F) (linear regression in a feature space)

$$g(\boldsymbol{x}) = w_1x_1^2 + w_2x_2^2 + w_3\sqrt{2}x_1x_2$$

- Inner product in the feature space

$$<\phi(\boldsymbol{x}), \phi(\boldsymbol{z})> = <(x_1^2, x_2^2, \sqrt{2}x_1x_2), (z_1^2, z_2^2, \sqrt{2}z_1z_2)>$$

$$= x_1^2z_1^2 + x_2^2z_2^2 + 2x_1x_2z_1z_2 = (x_1z_1 + x_2z_2)^2 = <\boldsymbol{x}, \boldsymbol{z}>^2$$

$$\mathrm{K}(\boldsymbol{x}, \boldsymbol{z}) = <\boldsymbol{x}, \boldsymbol{z}>^2$$

x_1 first feature and x_2 second feature

<> inner product

Shawe-Taylor, John, and Nello Cristianini. Kernel methods for pattern analysis. Cambridge university press, 2004.

Input space to feature space

$K(\boldsymbol{x}, \boldsymbol{z}) = < \boldsymbol{x}, \boldsymbol{z} >^2 = (x_1 z_1 + x_2 z_2)^2$

$\boldsymbol{x} = (2, 4)$
$\boldsymbol{z} = (3, 5)$

Calculate $< \phi(\boldsymbol{x}), \phi(\boldsymbol{z}) >$. Here, $\phi(\boldsymbol{x}) = (x_1^2, x_2^2, \sqrt{2}\, x_1 x_2)$

Without kernel $F(\boldsymbol{x}).F(\boldsymbol{z}) = (x_1^2, x_2^2, \sqrt{2} x_1 x_2).\ (z_1^2, z_2^2, \sqrt{2}\, z_1 z_2)$

$F(\boldsymbol{x}) = (x_1^2, x_2^2, \sqrt{2} x_1 x_2) = (2^2, 4^2, \sqrt{2}\ 2 \text{ x } 4) = (4, 16, 8\sqrt{2})$

$F(\boldsymbol{z}) = (z_1^2, z_2^2, \sqrt{2} z_1 z_2) = (3^2, 5^2, \sqrt{2}\ 3 \text{ x } 5) = (9, 25, 15\sqrt{2})$

$F(\boldsymbol{x}).\ F(\boldsymbol{z}) = 4\text{x}9 + 16\text{x}25 + 8\sqrt{2} \text{ x } 15\sqrt{2} = 676$

Input space to feature space

$K(\boldsymbol{x}, \boldsymbol{z}) = <\boldsymbol{x}, \boldsymbol{z}>^2 = (x_1 z_1 + x_2 z_2)^2$

$\boldsymbol{x} = (2,4)$
$\boldsymbol{z} = (3,5)$

Calculate $<\phi(\boldsymbol{x}), \phi(\boldsymbol{z})>$. Here, $\phi(\boldsymbol{x}) = (x_1^2, x_2^2, \sqrt{2}x_1 x_2)$

With kernel $F(\boldsymbol{x}).F(\boldsymbol{z}) = (x_1^2, x_2^2, \sqrt{2}x_1 x_2).(z_1^2, z_2^2, \sqrt{2}\, z_1 z_2) = (x_1 z_1 + x_2 z_2)^2$

$K(\boldsymbol{x}, \boldsymbol{z}) = <\boldsymbol{x}, \boldsymbol{z}>^2 = (x_1 z_1 + x_2 z_2)^2 = (2 \text{ x } 3 + 4 \text{ x } 5)^2 = 676$

No need to calculate individual $F(\boldsymbol{x})$ and $F(\boldsymbol{z})$

Input space to feature space

- Three − dimensional input space $\boldsymbol{x} \subseteq \mathbb{R}^3$ with feature map

Without kernel

$$(x_1 z_1)^2 + (x_2 z_2)^2 + (x_3 z_3)^2 +$$

$$2(x_1 z_1)(x_2 z_2) + 2(x_1 z_1)(x_3 z_3) +$$

$$2(x_2 z_2)(x_3 z_3)$$

With kernel

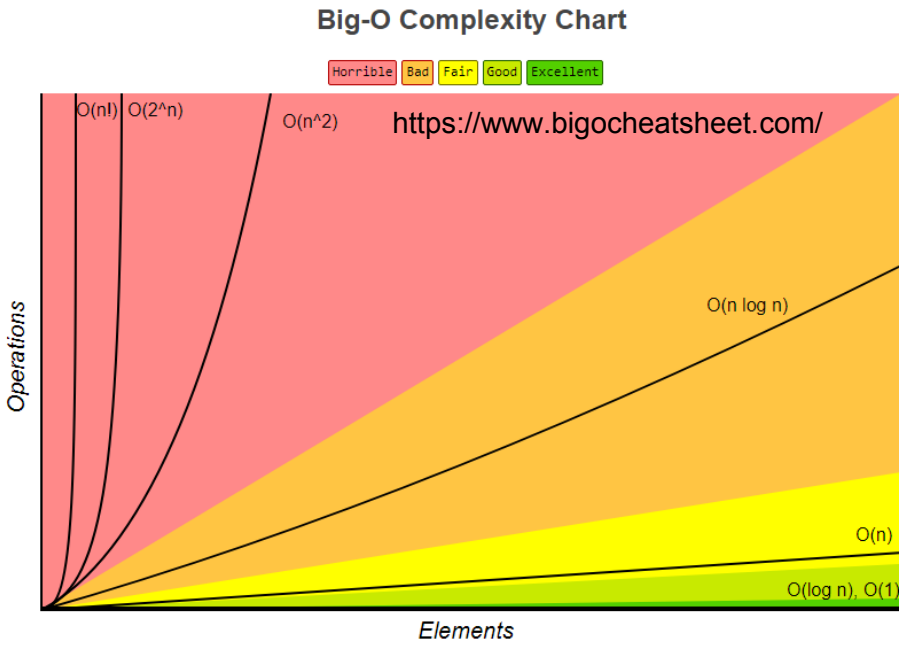
$$K(\boldsymbol{x}, \boldsymbol{z}) = <\boldsymbol{x}, \boldsymbol{z}>^2 = (\boldsymbol{x}^T \boldsymbol{z})^2 = (x_1 z_1 + x_2 z_2 + x_3 z_3)^2$$

polynomial kernel

Input space to feature space

- $K(\boldsymbol{x}, \boldsymbol{z}) = (\boldsymbol{x}^T\boldsymbol{z} + c)^{\boldsymbol{d}}$, $\boldsymbol{x}$ and $\boldsymbol{z} \in \mathbb{R}^n, C \in \mathbb{R}$
- O(n) – computational oppose to without kernel it is $O(n^2)$

e.g., n=3



x_1x_1
x_1x_2
x_1x_3
x_2x_1
x_2x_2
x_2x_3
x_3x_1
x_3x_2
x_3x_3
$\sqrt{2c}\,x_1$
$\sqrt{2c}\,x_2$
$\sqrt{2c}\,x_3$
c

$O(n^d)$ feature space only need O(n) operations

e.g., n=3 and d=2 , O(6) feature space needs O(3) computations

Valid kernel function

- For a function to be a valid kernel, its Gram matrix **K** must be positive semidefinite [1].
- i.e., **K**'s elements, given by $\boldsymbol{K}(\mathbf{x}_i, \mathbf{x}_j)$ should be symmetric positive semidefinite (PSD)* for all $\{\mathbf{x}_i\}$

(Mercer's theorem)

$$\boldsymbol{K} = \begin{bmatrix} K(\boldsymbol{x}_1, \boldsymbol{x}_1) & K(\boldsymbol{x}_1, \boldsymbol{x}_2) & \cdots & K(\boldsymbol{x}_1, \boldsymbol{x}_N) \\ K(\boldsymbol{x}_2, \boldsymbol{x}_1) & K(\boldsymbol{x}_2, \boldsymbol{x}_2) & & K(\boldsymbol{x}_2, \boldsymbol{x}_N) \\ \vdots & & \ddots & \vdots \\ K(\boldsymbol{x}_N, \boldsymbol{x}_1) & K(\boldsymbol{x}_N, \boldsymbol{x}_2) & \cdots & K(\boldsymbol{x}_N, \boldsymbol{x}_N) \end{bmatrix}$$

Gram matrix
(Kernel Matrix)

*A matrix is positive semidefinite if and only if all of its eigenvalues are nonnegative or greater than or equal to zero. Positive semidefinite isn't the same as having nonnegative elements in the matrix. A matrix with nonnegative elements can have a mix of positive, zero, and negative eigenvalues.

[1] Shawe-Taylor, John, and Nello Cristianini. Kernel methods for pattern analysis. Cambridge university press, 2004.

Kernels

- Linear kernel

$$K(\boldsymbol{x}, \boldsymbol{z}) = (\boldsymbol{x}^T \boldsymbol{z} + c)$$

$K(\boldsymbol{x}, \boldsymbol{z})$ is larger ➔ $\boldsymbol{x}$ and $\boldsymbol{z}$ are similar

- Polynomial kernel

$$K(\boldsymbol{x}, \boldsymbol{z}) = (\boldsymbol{x}^T \boldsymbol{z} + c)^{\boldsymbol{d}}$$

- Gaussian kernel

$$K(\boldsymbol{x}, \boldsymbol{z}) = \exp\left(\frac{-\|\boldsymbol{x}-\boldsymbol{z}\|_2^2}{2\sigma^2}\right)$$

$K(\boldsymbol{x}, \boldsymbol{z}) \to 1$, when $\boldsymbol{x}$ and $\boldsymbol{z}$ are close

$K(\boldsymbol{x}, \boldsymbol{z}) \to 0$, when $\boldsymbol{x}$ and $\boldsymbol{z}$ arefar apart

$c \geq 0$

Techniques for constructing new kernels

- Given two kernels $K_1(\boldsymbol{x}, \boldsymbol{z})$ and $K_2(\boldsymbol{x}, \boldsymbol{z})$

$K(\boldsymbol{x}, \boldsymbol{z}) = c\, K_1(\boldsymbol{x}, \boldsymbol{z})$, for any constant $c > 0$

$K(\boldsymbol{x}, \boldsymbol{z}) = K_1(\boldsymbol{x}, \boldsymbol{z}) + K_2(\boldsymbol{x}, \boldsymbol{z})$

$K(\boldsymbol{x}, \boldsymbol{z}) = K_1(\boldsymbol{x}, \boldsymbol{z})\, K_2(\boldsymbol{x}, \boldsymbol{z})$

$K(\boldsymbol{x}, \boldsymbol{z}) = \exp(K_1(\boldsymbol{x}, \boldsymbol{z}))$

$K(\boldsymbol{x}, \boldsymbol{z}) = f(\boldsymbol{x})\, K_1(\boldsymbol{x}, \boldsymbol{z})\, f(\boldsymbol{z})$, for any function f

$K(\boldsymbol{x}, \boldsymbol{z}) = q(K_1(\boldsymbol{x}, \boldsymbol{z}))$, for any polynomial q with nonnegative coefficients

$K(\boldsymbol{x}, \boldsymbol{z}) = \boldsymbol{x}^T \boldsymbol{A} \boldsymbol{z}$, for any symmetric psd matrix **A**

C. M. Bishop, Pattern Recognition and Machine Learning, Springer NY, 2006

Estimating the kernel

- Many kernels typically include parameters that can significantly influence the model's predictions.

$$\mathrm{K}(\boldsymbol{x}, \boldsymbol{z}) = \exp\left(\frac{-\|\boldsymbol{x}-\boldsymbol{z}\|_2^2}{2\sigma^2}\right)$$

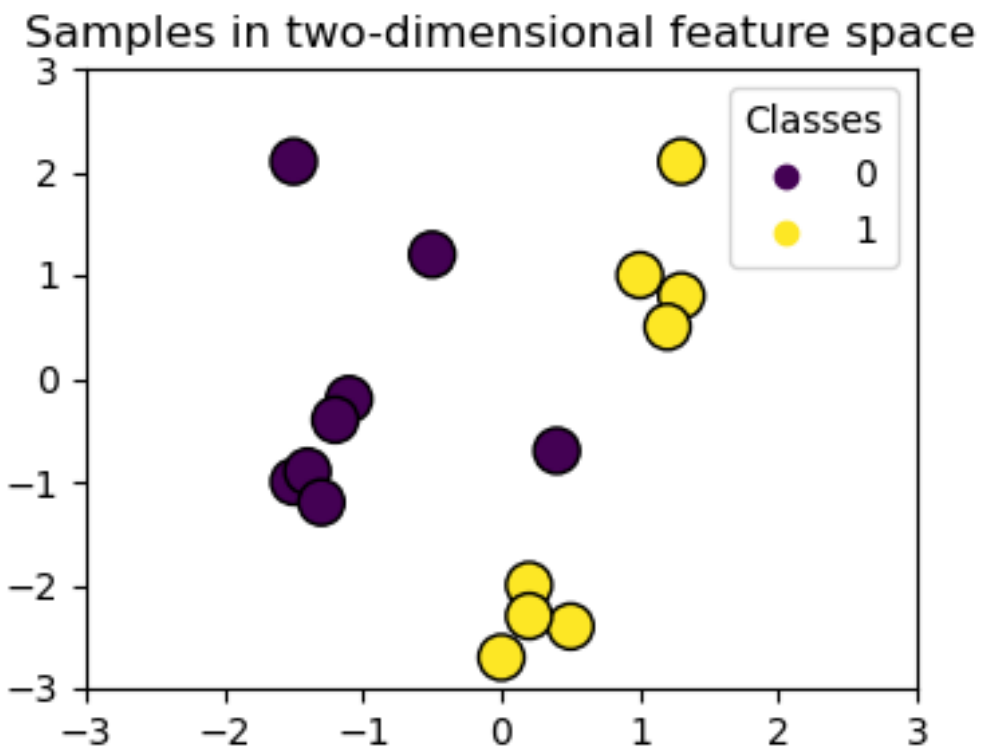
- **Grid Search:** Performing an exhaustive search over a discrete grid of values and selecting a value which provides the highest model performance. Can be slow.
- **Empirical Bayes:** Maximize the marginal likelihood and typically utilize gradient-based optimization techniques, which offer faster convergence.
- **Bayesian inference:** Bayesian statistical methods to estimate or learn the appropriate kernel parameters. Useful when few data samples are available.

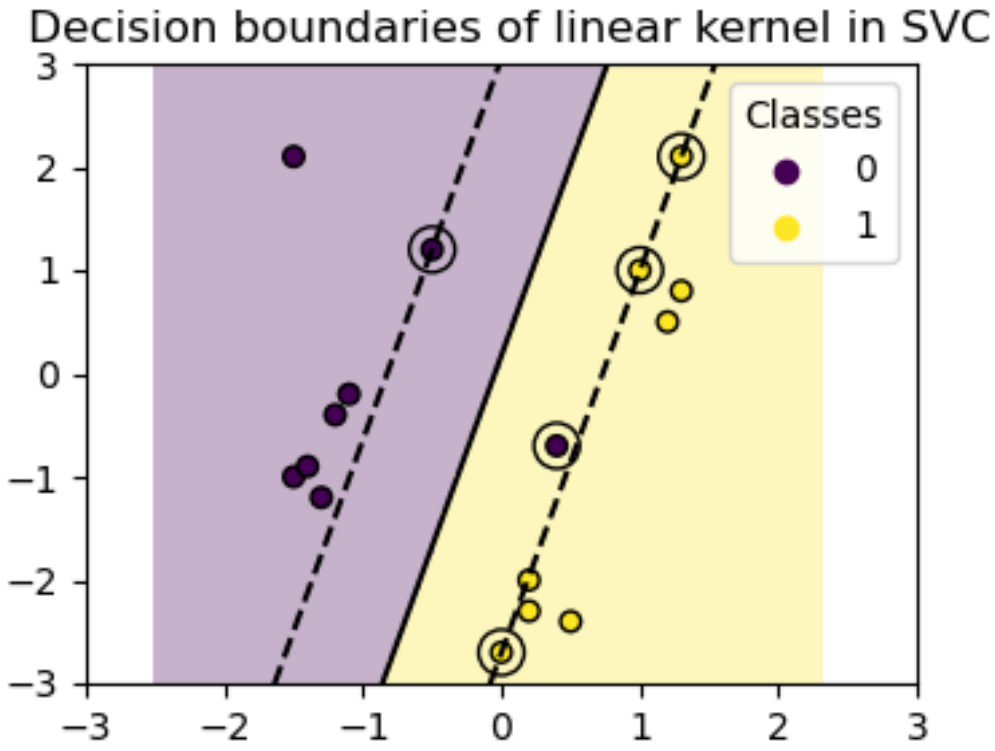
Examples

- Plot classification boundaries with different SVM Kernels
- SVM with custom kernel
- Harvard CS109A | Support Vector Machine Demo

Examples

- Plot classification boundaries with different SVM Kernels
  https://scikit-learn.org/stable/auto_examples/svm/plot_svm_kernels.html

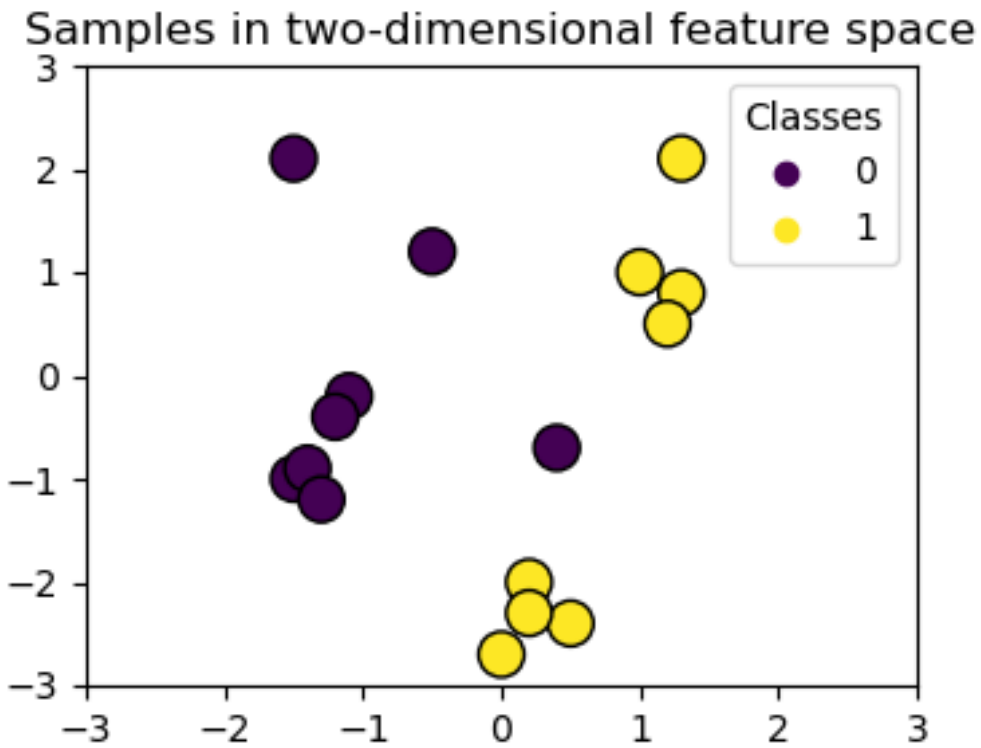


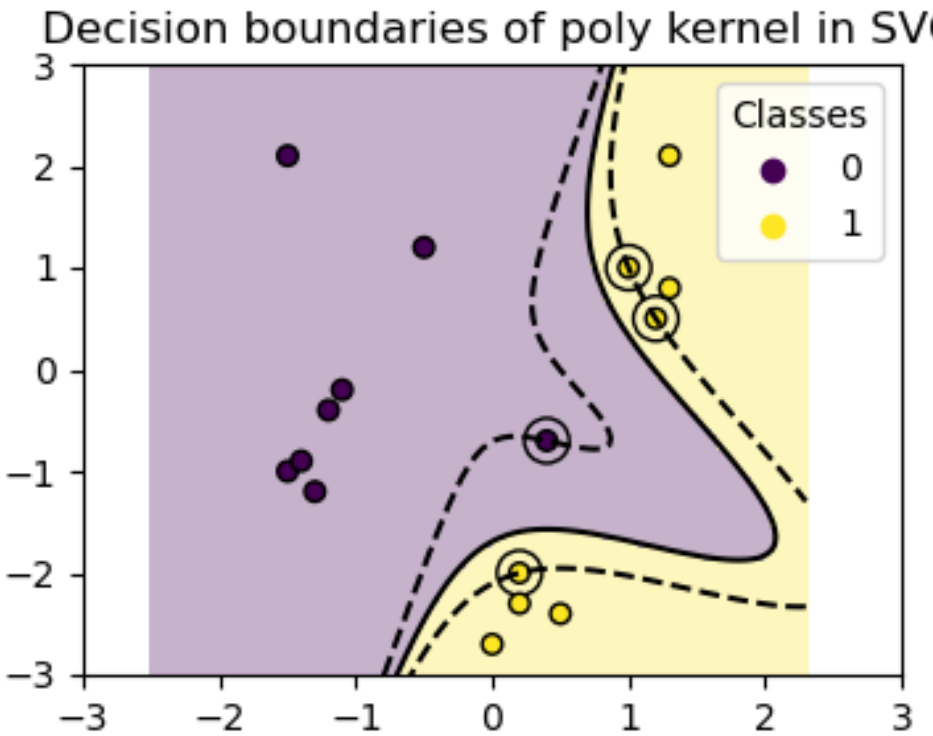


$$K(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1^\top \mathbf{x}_2$$

Examples

- Plot classification boundaries with different SVM Kernels
  https://scikit-learn.org/stable/auto_examples/svm/plot_svm_kernels.html





$d = 3$
$r = 0$
$\gamma = 2$

$$K(\mathbf{x}_1, \mathbf{x}_2) = (\gamma \cdot \mathbf{x}_1^\top \mathbf{x}_2 + r)^d$$

Examples: Polynomial Kernel

$$K(\mathbf{x}_1, \mathbf{x}_2) = (\gamma \cdot \mathbf{x}_1^\top \mathbf{x}_2 + r)^d$$

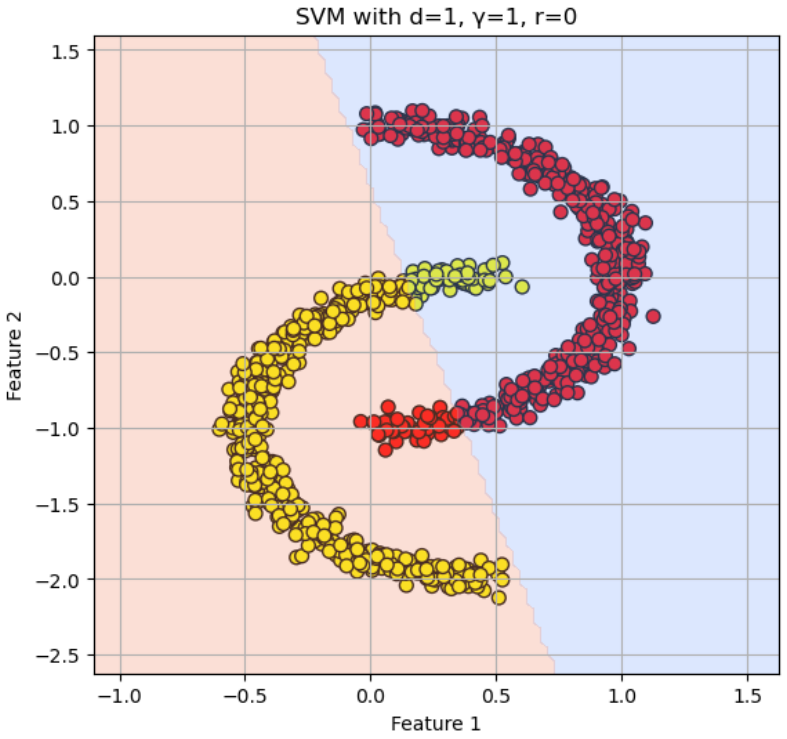
d- Degree of the Polynomial: determines the complexity of the decision boundary

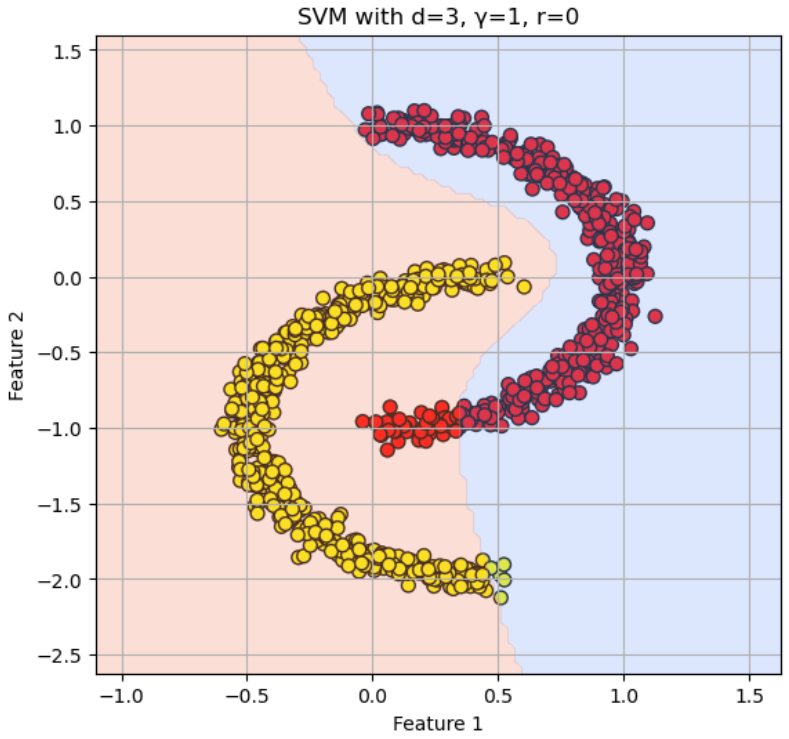
r-bias: shifts the data up or down

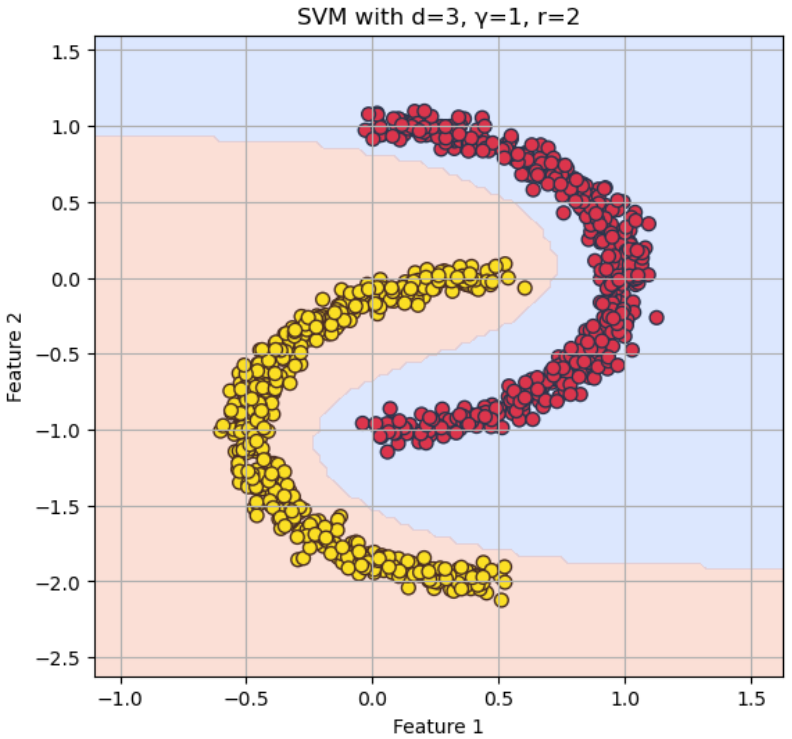
γ: controls the influence of individual training sample

Small values of γ mean far points have a greater influence, leading to smoother decision boundaries.
Large values of γ mean only nearby points influence the decision boundary, allowing for more complex shapes.

Examples: Polynomial Kernel

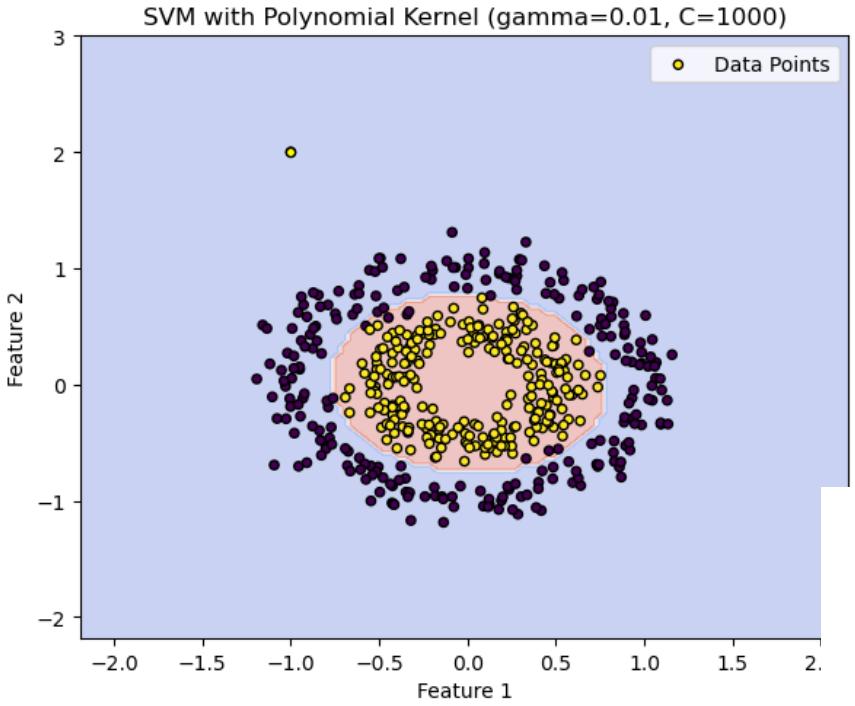


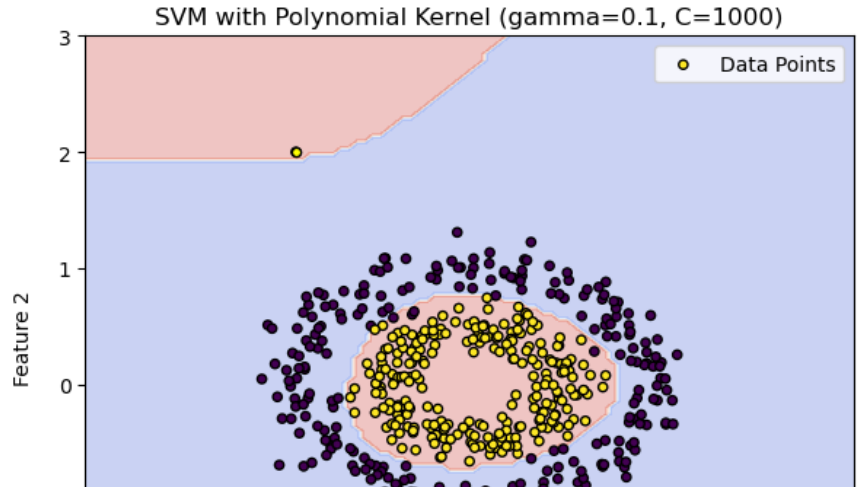


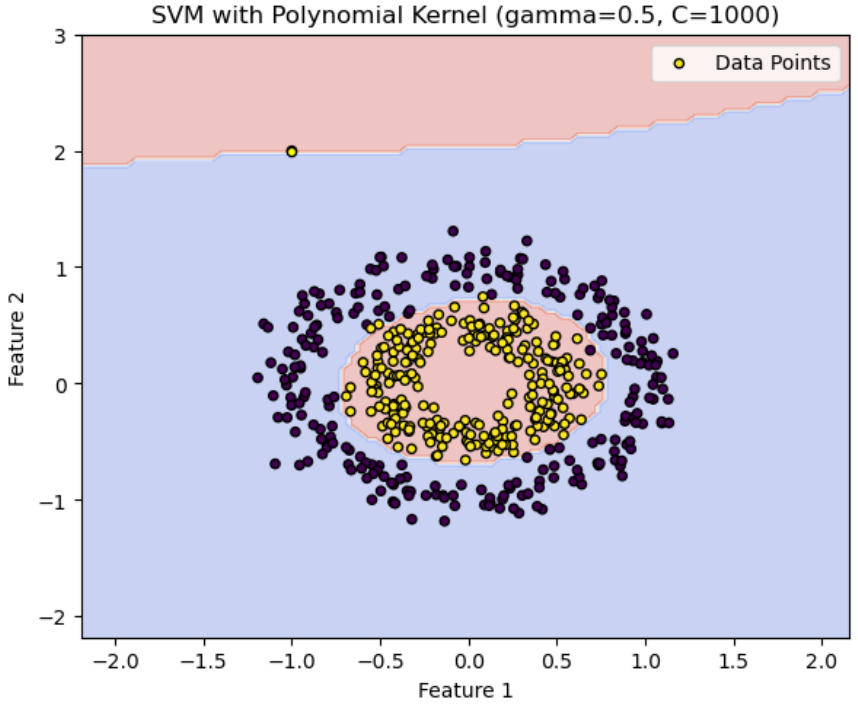


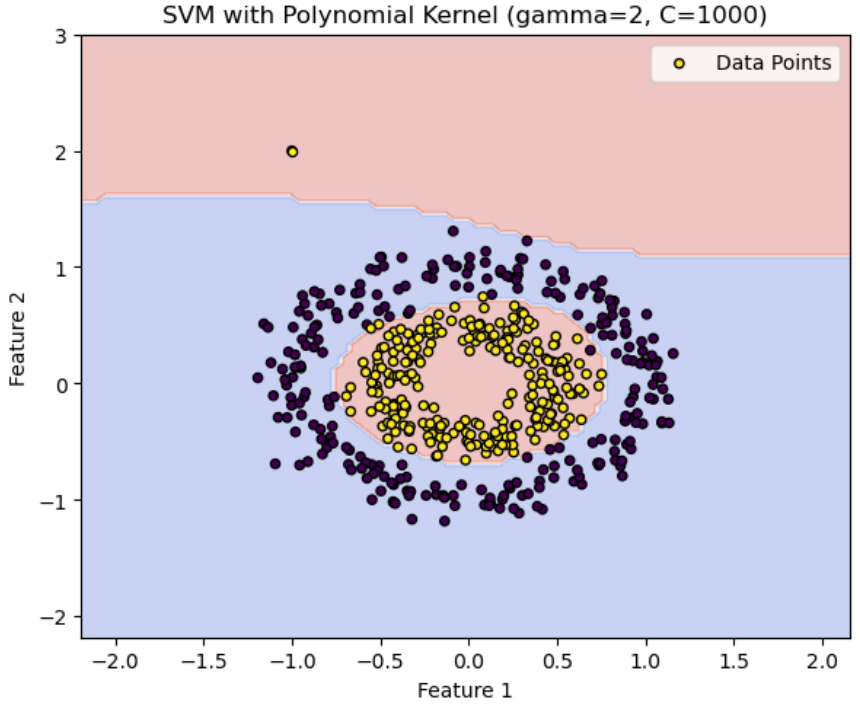
$$K(\mathbf{x}_1, \mathbf{x}_2) = (\gamma \cdot \mathbf{x}_1^\top \mathbf{x}_2 + r)^d$$

Examples: Polynomial Kernel









$$K(\mathbf{x}_1, \mathbf{x}_2) = (\gamma \cdot \mathbf{x}_1^\top \mathbf{x}_2 + r)^d$$

Small values of γ mean far points have a greater influence. Large values of γ mean only nearby points influence the decision boundary.

Density Estimation

Based on Lecture 9: Density Estimation — Applied ML (kuleshov-group.github.io)

Density estimation

- Estimating the probability density function (PDF) of a random variable.

$$P_{data} \approx P_{\theta}$$

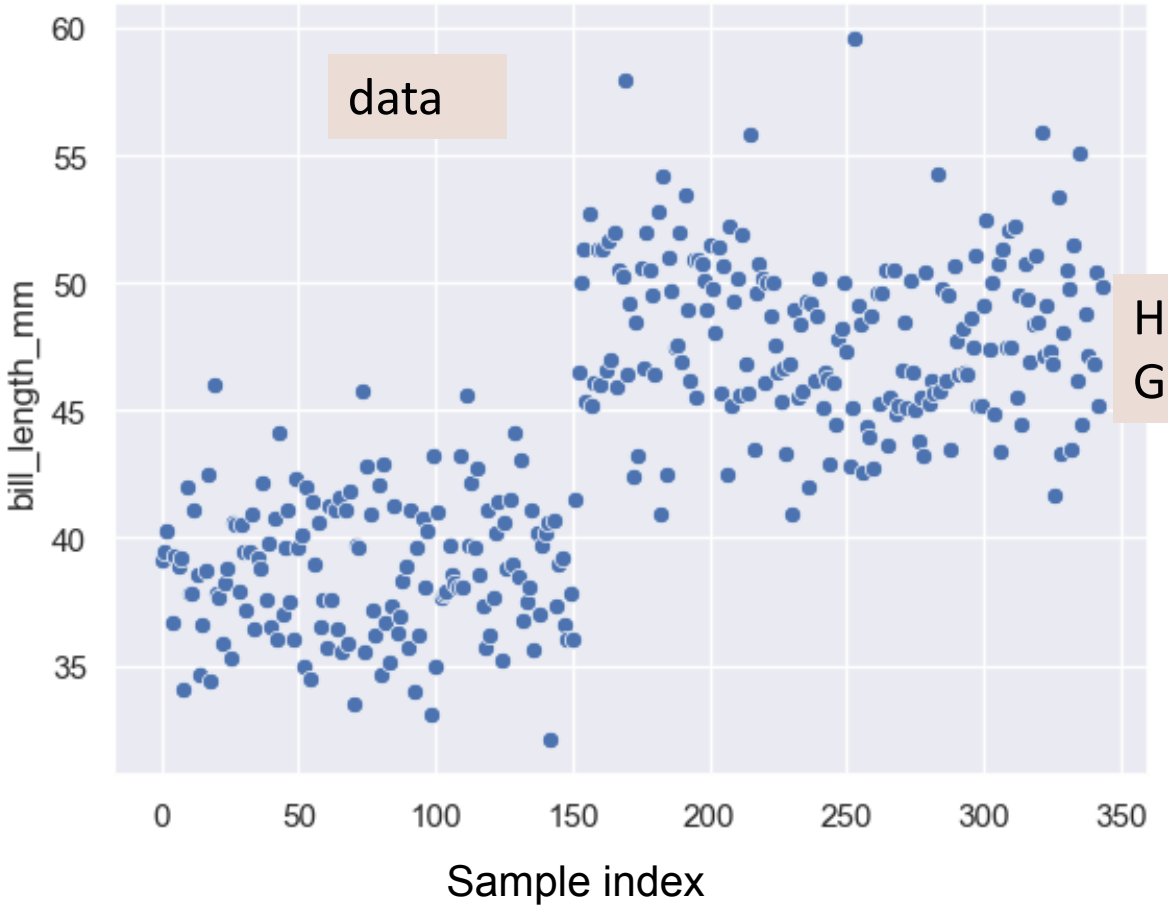
True distribution

Unknown

Estimated distribution

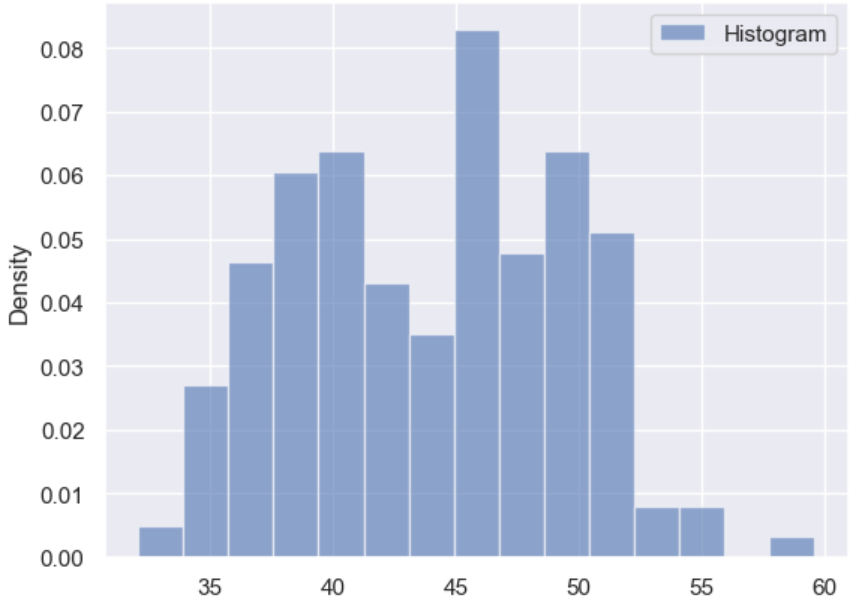
- After estimation of distribution
  - Data generation
  - Clustering
  - Outlier detection
  - Missing data filling
  - Vector quantization (short code can be assigned to higher probable data samples)

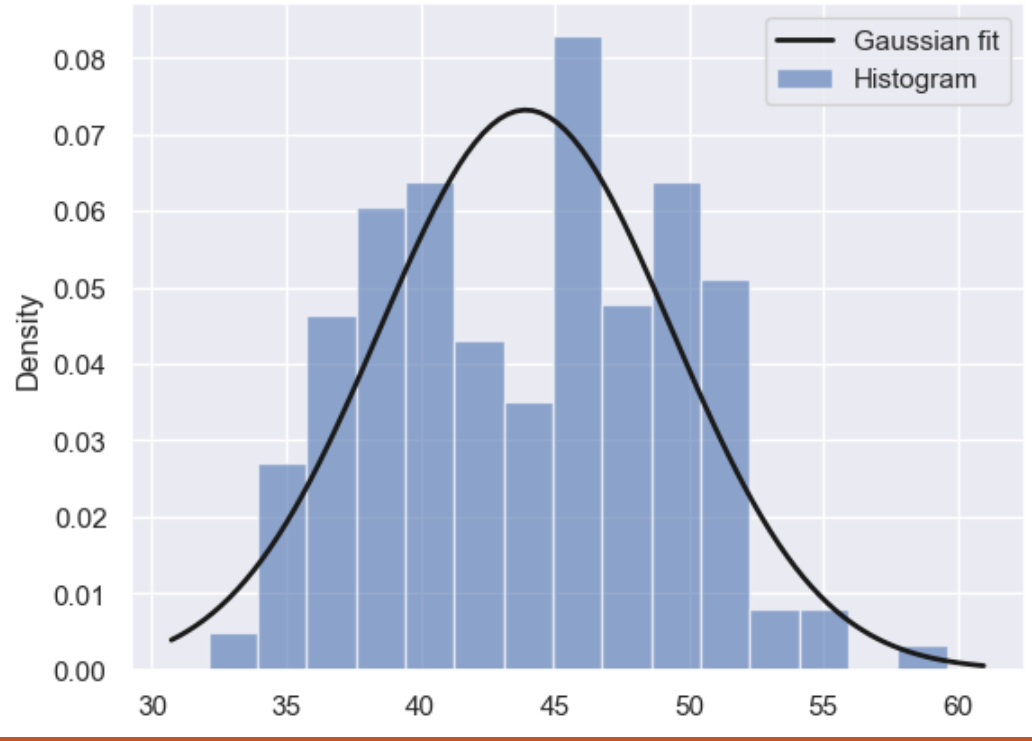
Density estimation



Histogram

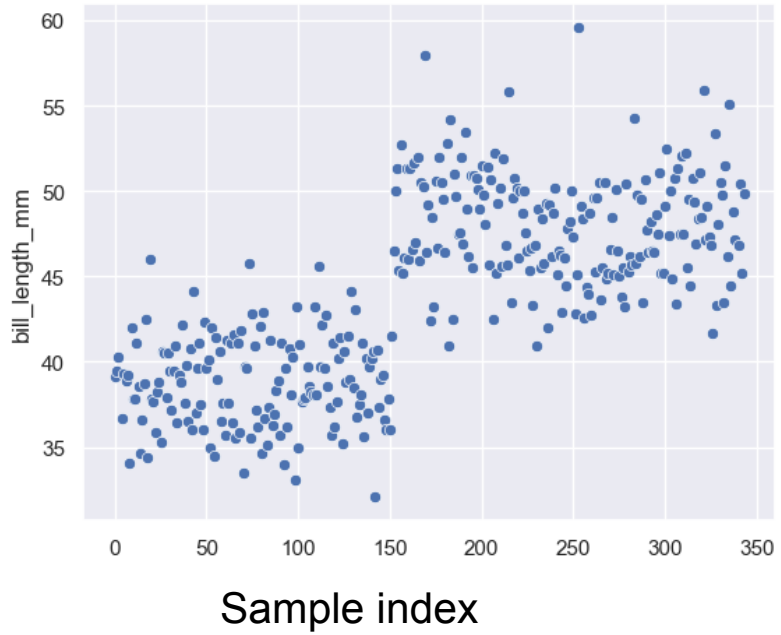
Histogram
Gaussian fit

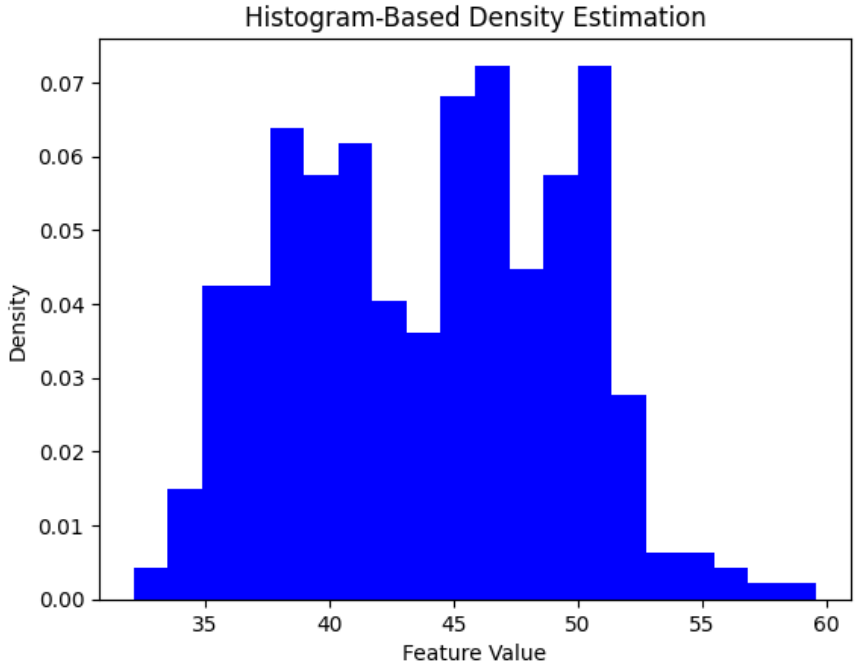




Density estimation

Histogram: A histogram divides the data into discrete bins and counts the number of data points that fall into each bin. It provides a discrete approximation of the data's probability density.





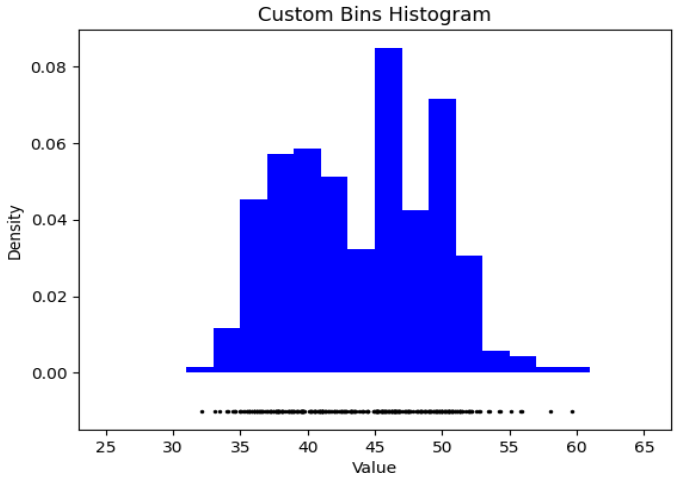
$P_\theta(x) \propto$ no of points in the bin where x falls

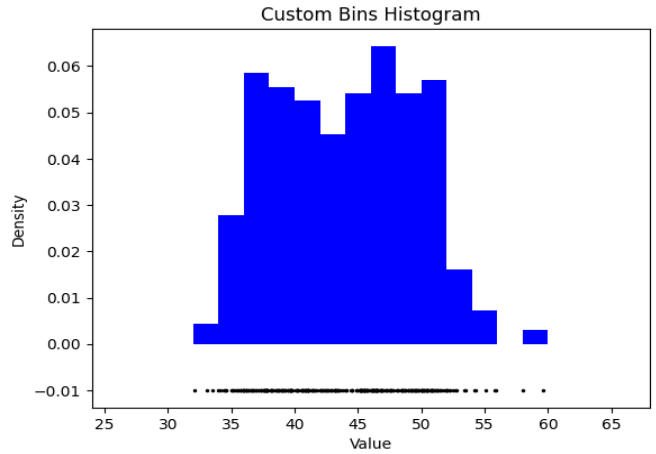
Density estimation

Histogram

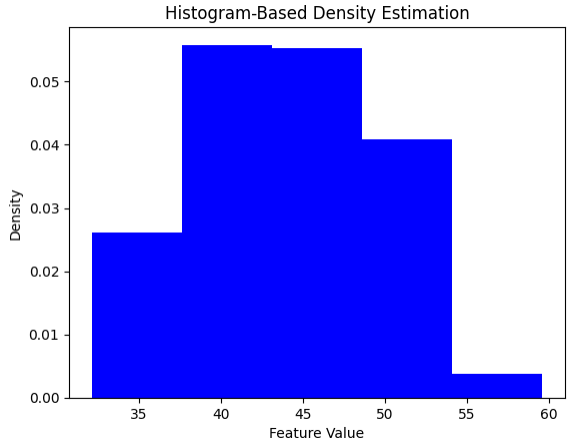
- Histogram not smooth
- Shape of the histogram depend on the **bin width** and **bin position**

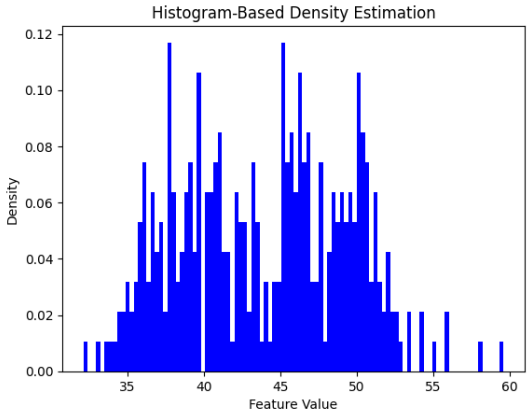
Kernel density estimation (KDE) address some of the issues of histogram density estimation





Same bin size but different starting positions





Different bin sizes

Kernal density estimation (KDE)

- Kernal density estimation (KDE)
- Statistical technique (non-parametric) used for estimating the probability density function of a random variable.
- Histogram: $P_\theta(x) \propto$ no of points in the bin where x falls
- KDE: $P_\theta(x) \propto$ no of points in dataset D (unsupervised) near $to\ x$
- Performance not depend on the location of the bin

e.g., Tophat Kernel Density Estimation

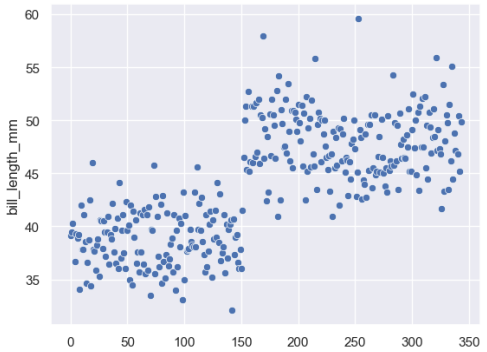
$$p_\delta(x) = \frac{N(x;\delta)}{n}$$

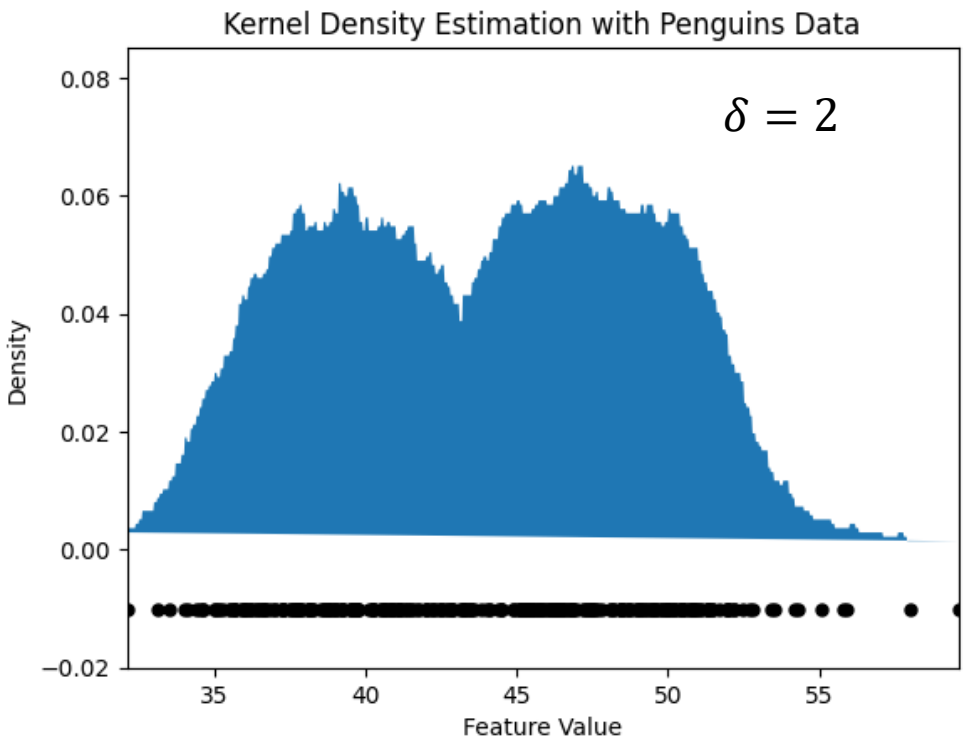
$$N(x;\delta) = \left|\{x^{(i)} : \left||x^{(i)} - x|\right| \leq \delta/2\ \}\right|$$

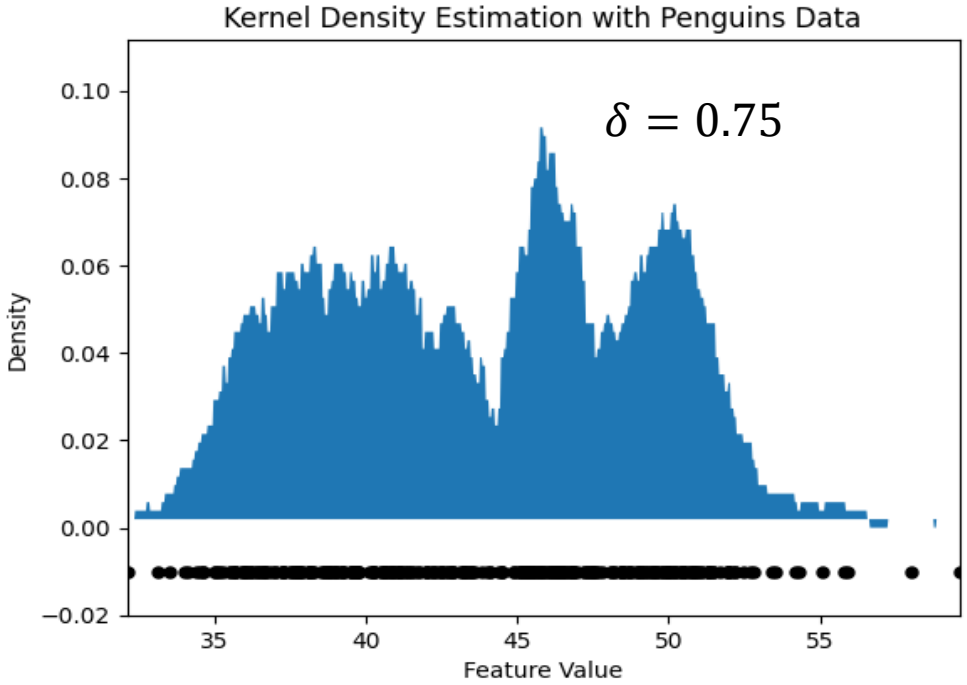
number of points that fall within a bin size of δ centered at x

n – total number of data points

Tophat Kernal density estimation (KDE)







- Performance not depend on the location of the bin
- Still density estimate is not smooth
- Possible solution: weighting

$$p_\delta(x) = \frac{N(x;\delta)}{n}$$

$$N(x;\delta) = \left|\{x^{(i)} : ||x^{(i)} - x|| \leq \delta/2\ \}\right|$$

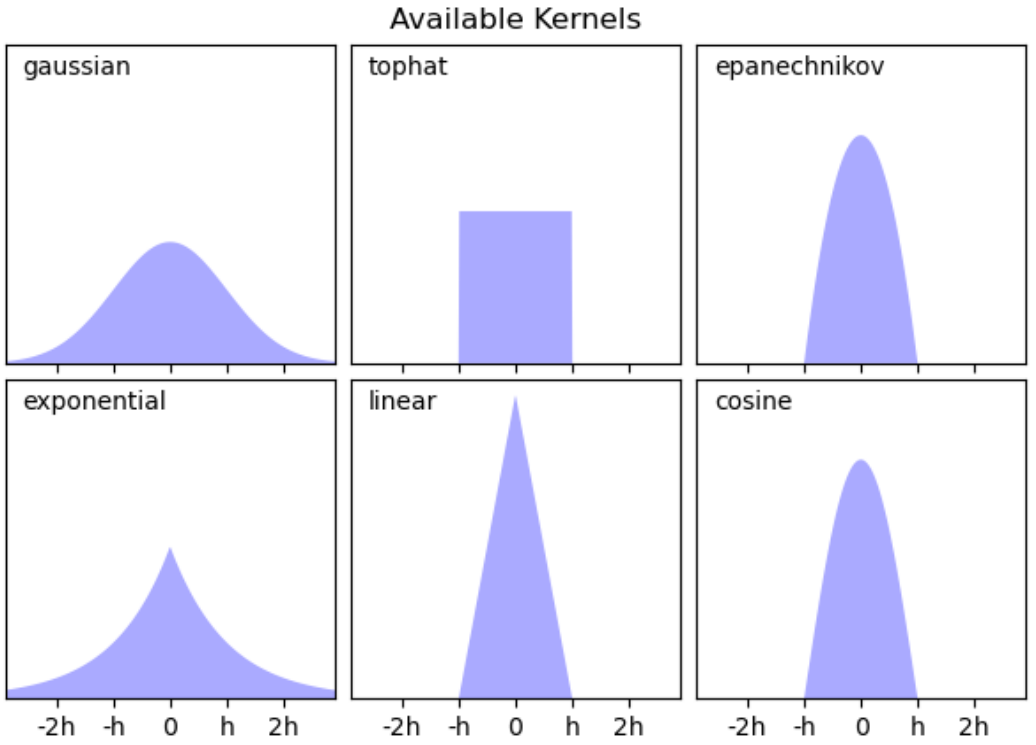
Weighting: $P_\theta\,(x) \propto$ no of points in D $near\ to\ x$, weighted by distance from x

Kernal density estimation (KDE)

- We count the points that are considered "close," with each point assigned a weight based on its similarity to another point

$$p(x) \propto \sum_{i=1}^{n} K(x, x^{(i)})$$

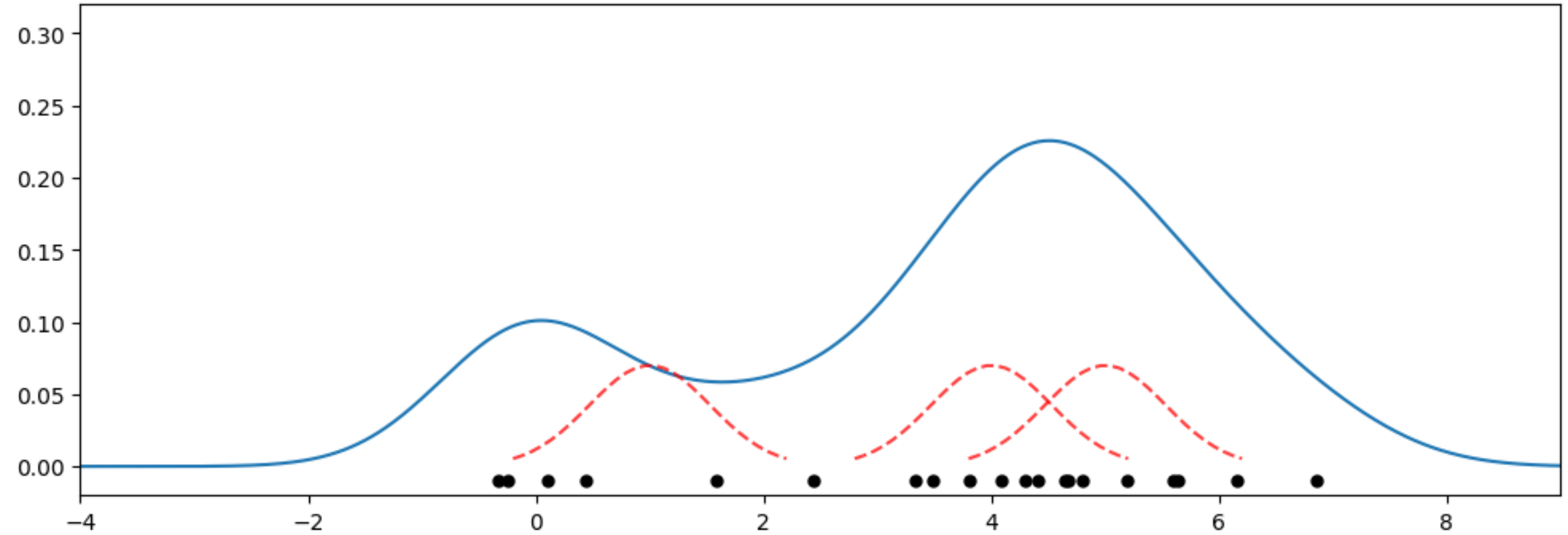
- Gaussian kernel $K(x;h) \propto \exp(-\frac{x^2}{2h^2})$
- Tophat kernel $K(x;h) \propto 1$ if $x < h$
- Epanechnikov kernel $K(x;h) \propto 1 - (\frac{x^2}{h^2})$
- Exponential kernel $K(x;h) \propto \exp(-x/h)$
- Linear kernel $K(x;h) \propto 1 - (x/h)$
- Cosine kernel $K(x;h) \propto \cos(\frac{\pi x}{2h})$ if $x < h$



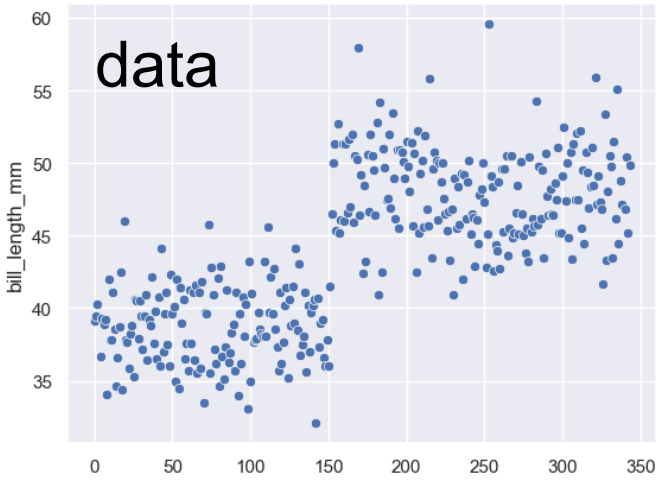
2.8. Density Estimation — scikit-learn 1.3.0 documentation

Kernal density estimation (KDE)

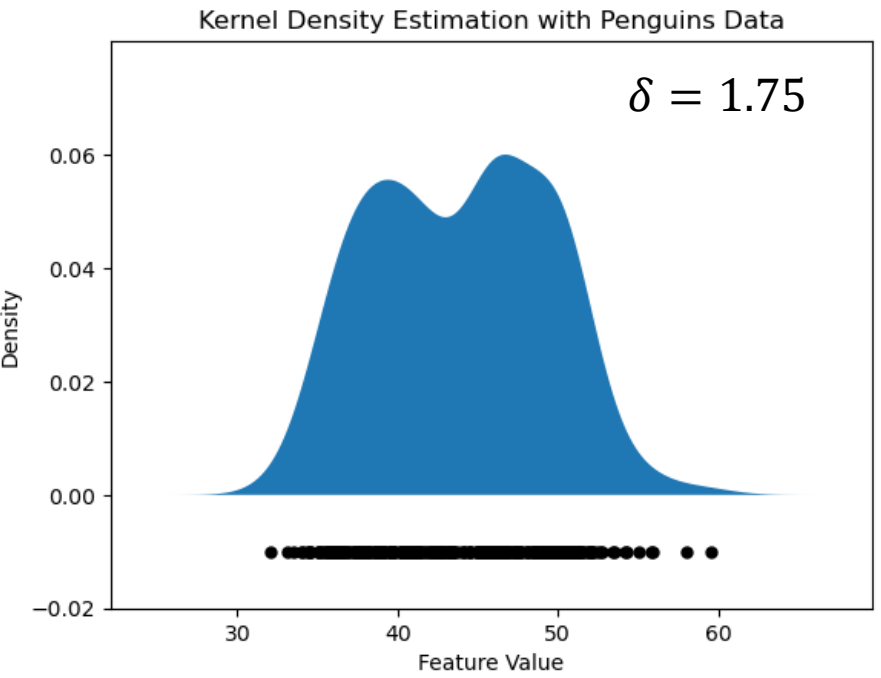
Gaussian example with individual Gaussians

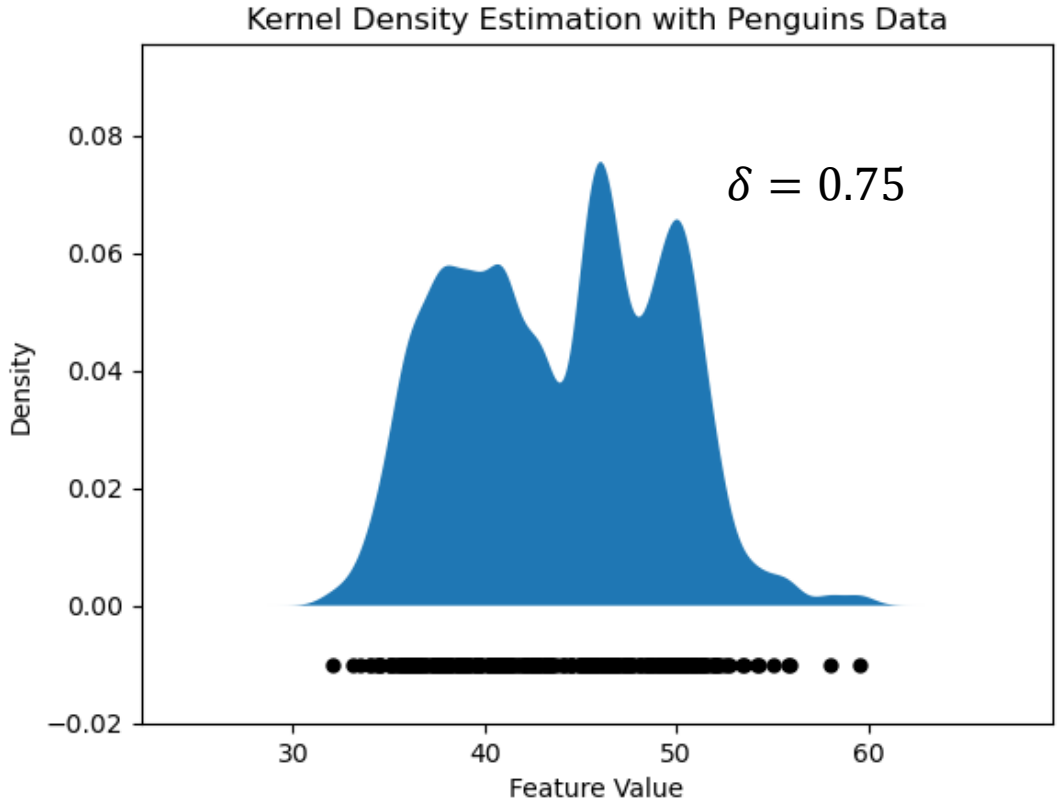


Kernal density estimation (KDE)



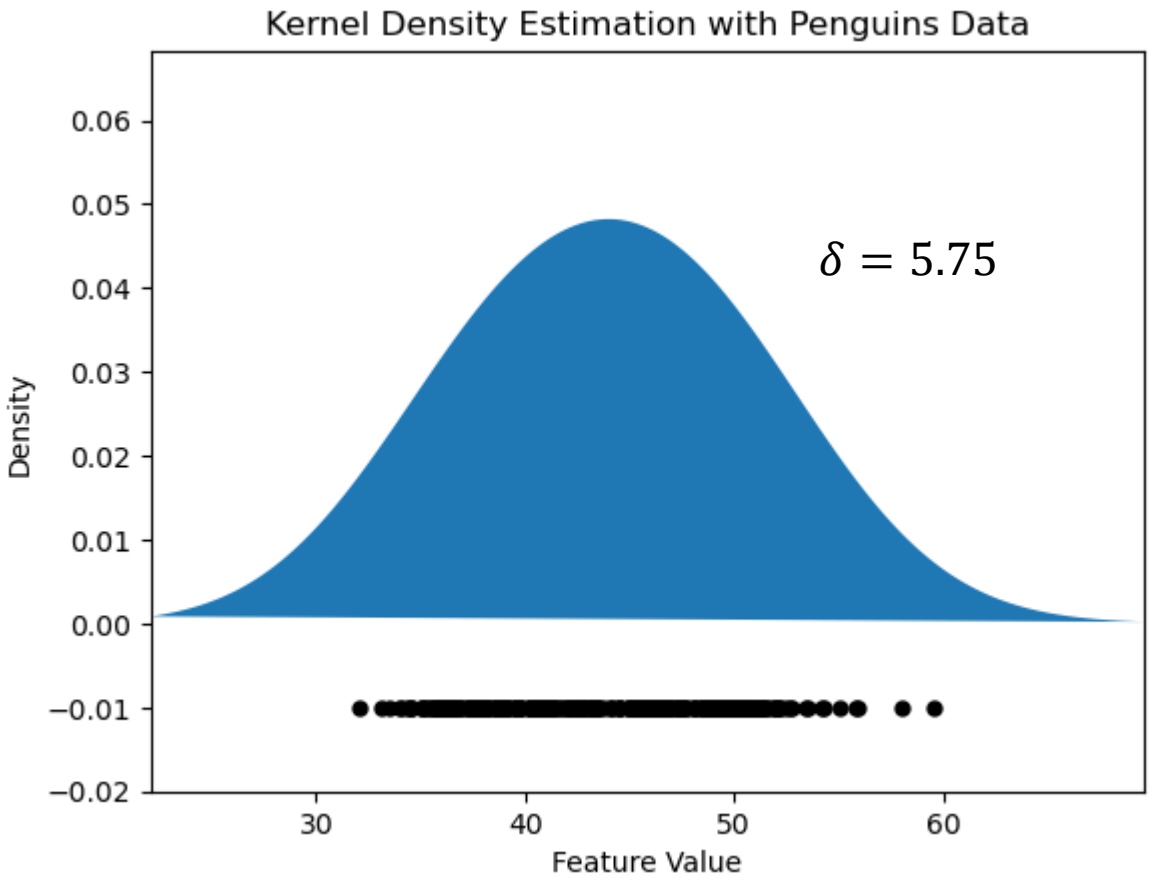
Gaussian density estimation

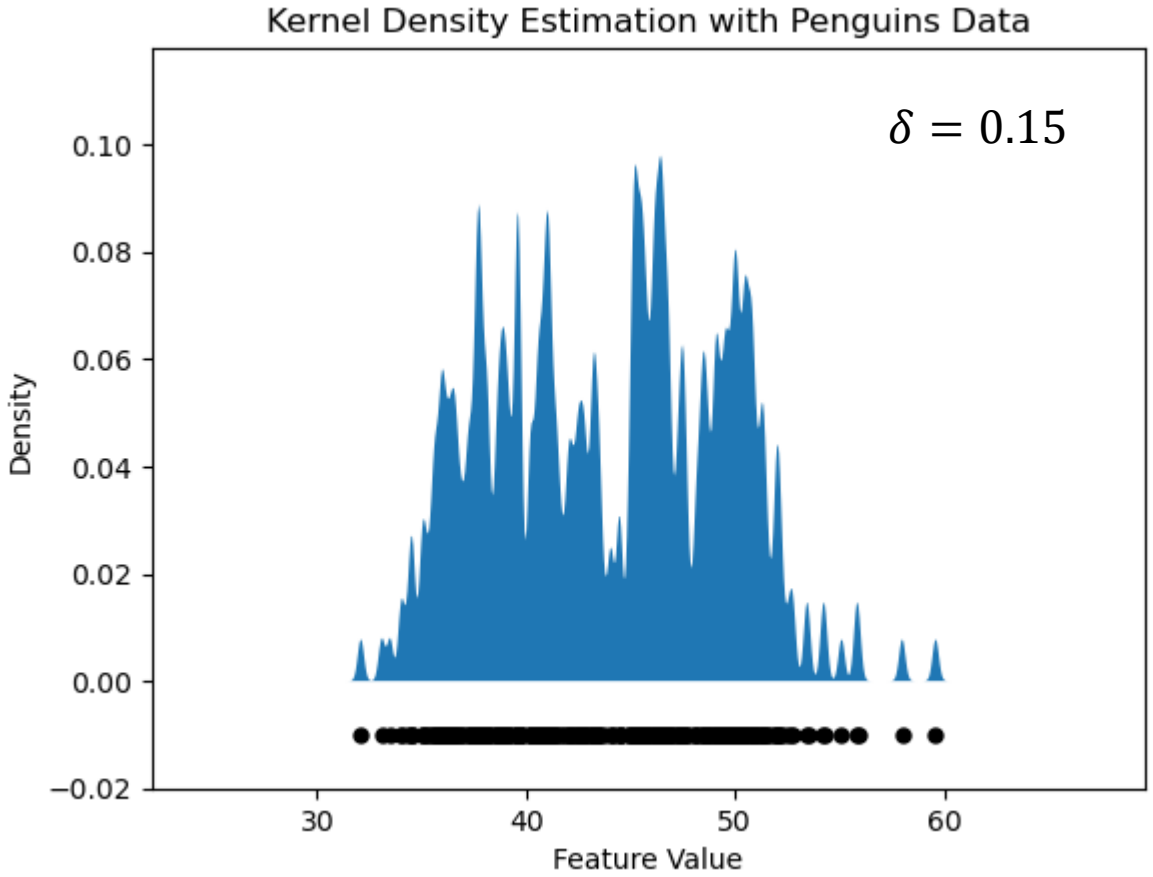




Kernal density estimation (KDE)

Gaussian:- impact of δ(bandwidth)





- The left figure is overly smoothed, leading to information loss.
- The right figure lacks smoothness and does not match the true distribution's shape.

Kernal density estimation (KDE)

- Heuristic methods: Scott's rule of thumb or Silverman's rule can be used to determine δ.
- Scott's rule of thumb $\delta \approx 1.06 \sqrt[5]{n}\, \sigma$ (assumes a unimodal (single-peaked) distribution)
- Silverman's rule $\delta \approx 0.9 \sqrt[5]{n} \min\left(\sigma, \frac{\mathrm{IQR}}{1.35}\right)$

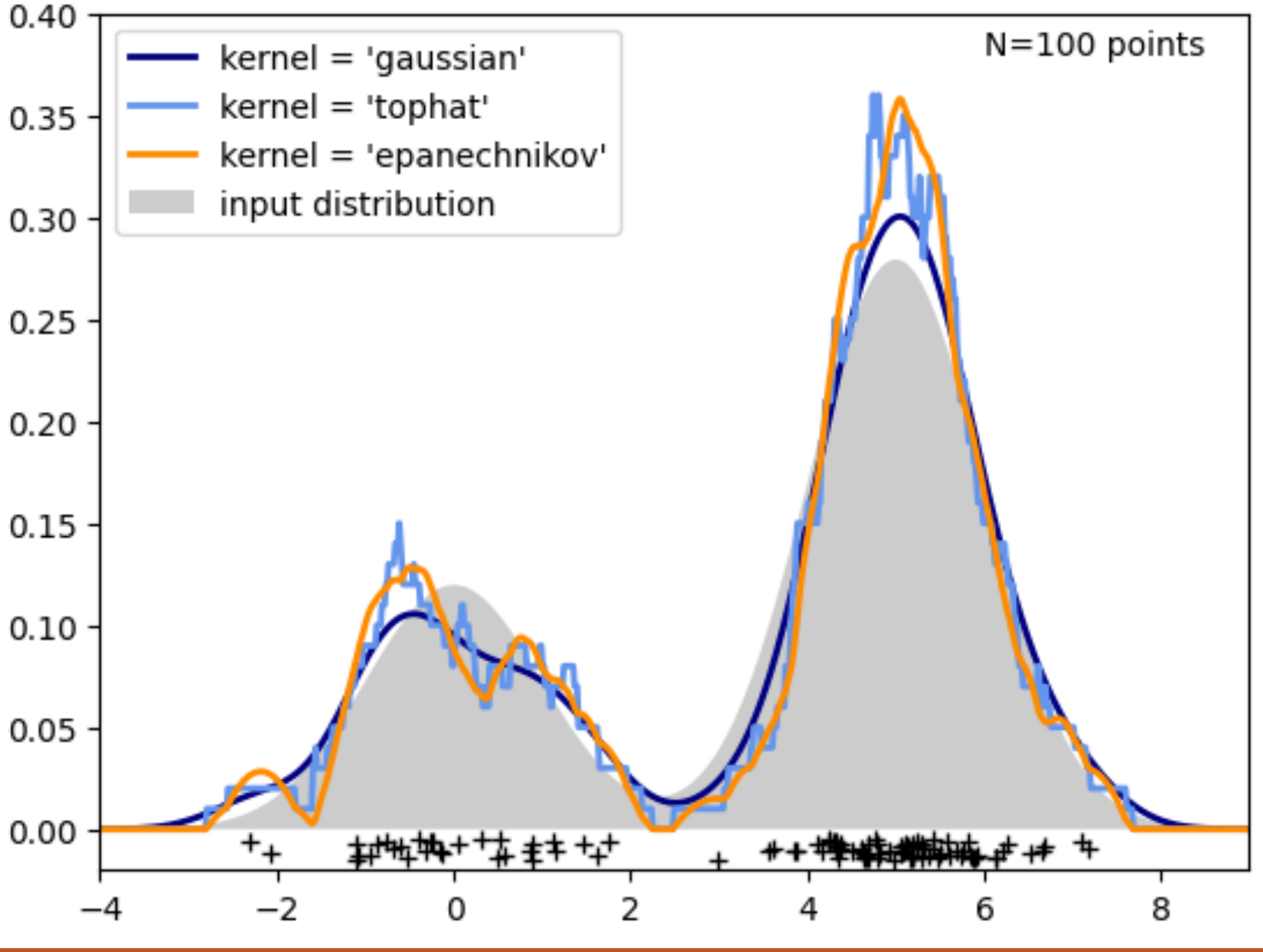
  (σ standard deviation of data , IQR^{*} -Interquartile range)
- Likelihood: By comparing likelihood values for different bandwidths, you can statistically assess which bandwidth provides highest likelihood.

*Interquartile range - Wikipedia

Kernel density estimation - Wikipedia

Silverman, B. W. (1986). Density Estimation. London: Chapman and Hall
Scott, D. W. (1992) Multivariate Density Estimation: Theory, Practice, and Visualization. New York: Wiley..

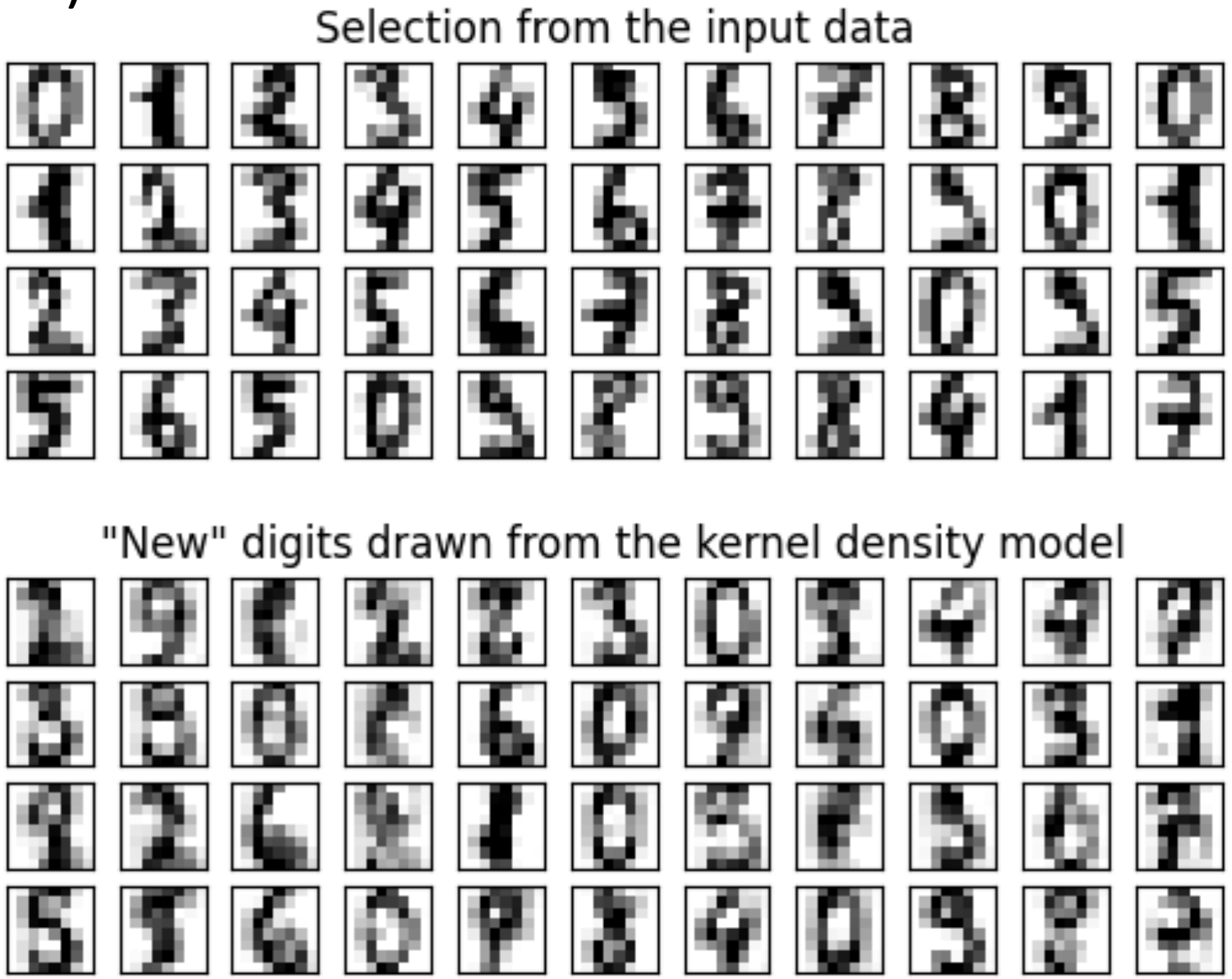
Kernel density estimation



Simple 1D Kernel Density Estimation — scikit-learn 1.3.0 documentation

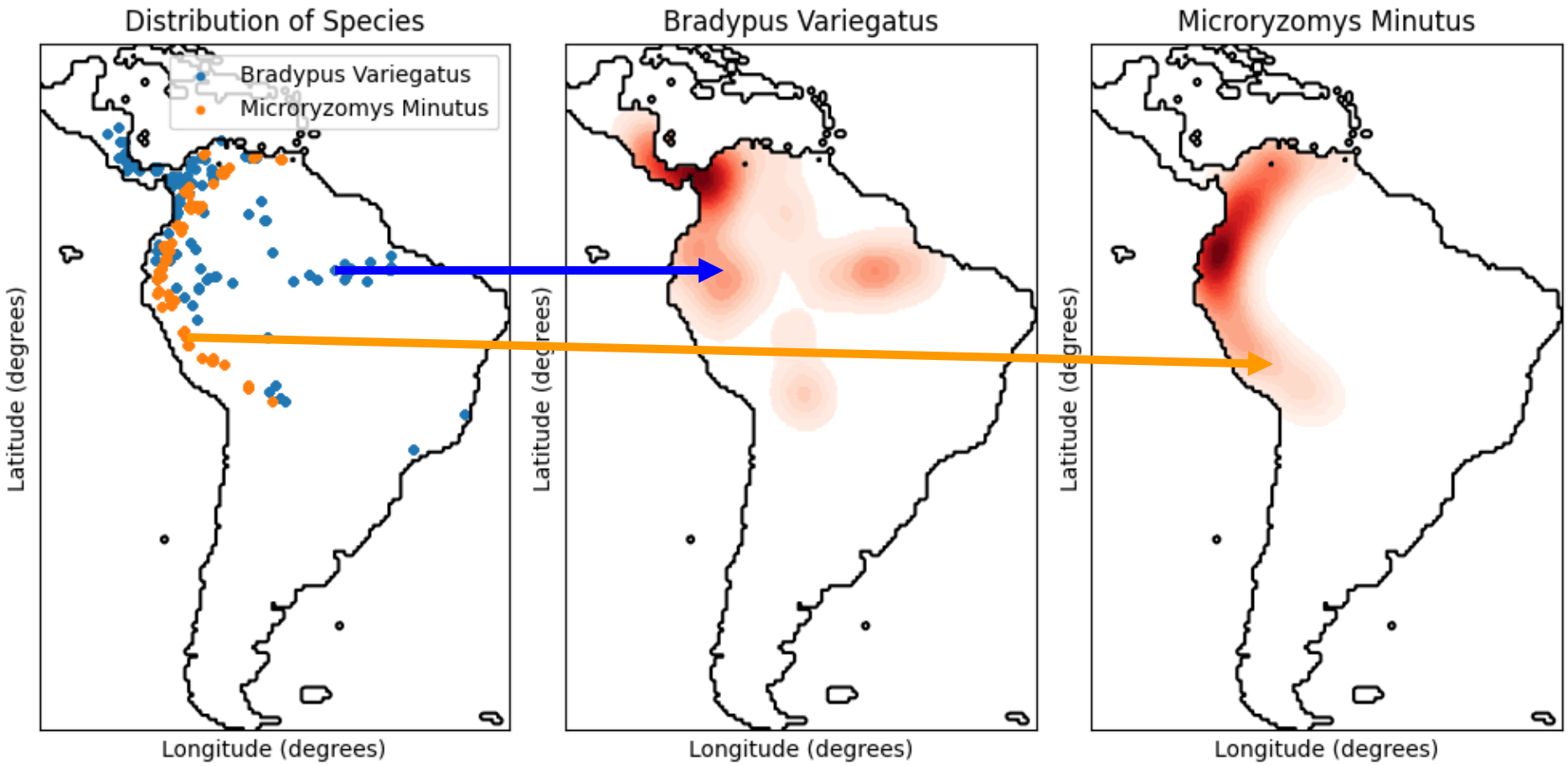
Kernal density estimation (KDE)

- New data generation from the estimated density.



Kernel Density Estimation — scikit-learn 1.3.0 documentation

Kernal density estimation (KDE)



- The kernel density estimation is used to visualize the distribution of two species on a map of South America.
- Two species, "Bradypus Variegatus" and "Microryzomys Minutus," are selected for analysis.

Kernel Density Estimate of Species Distributions — scikit-learn 1.3.0 documentation





Thank You

Q & A